



Arm[®] Architecture Reference Manual for A-profile architecture

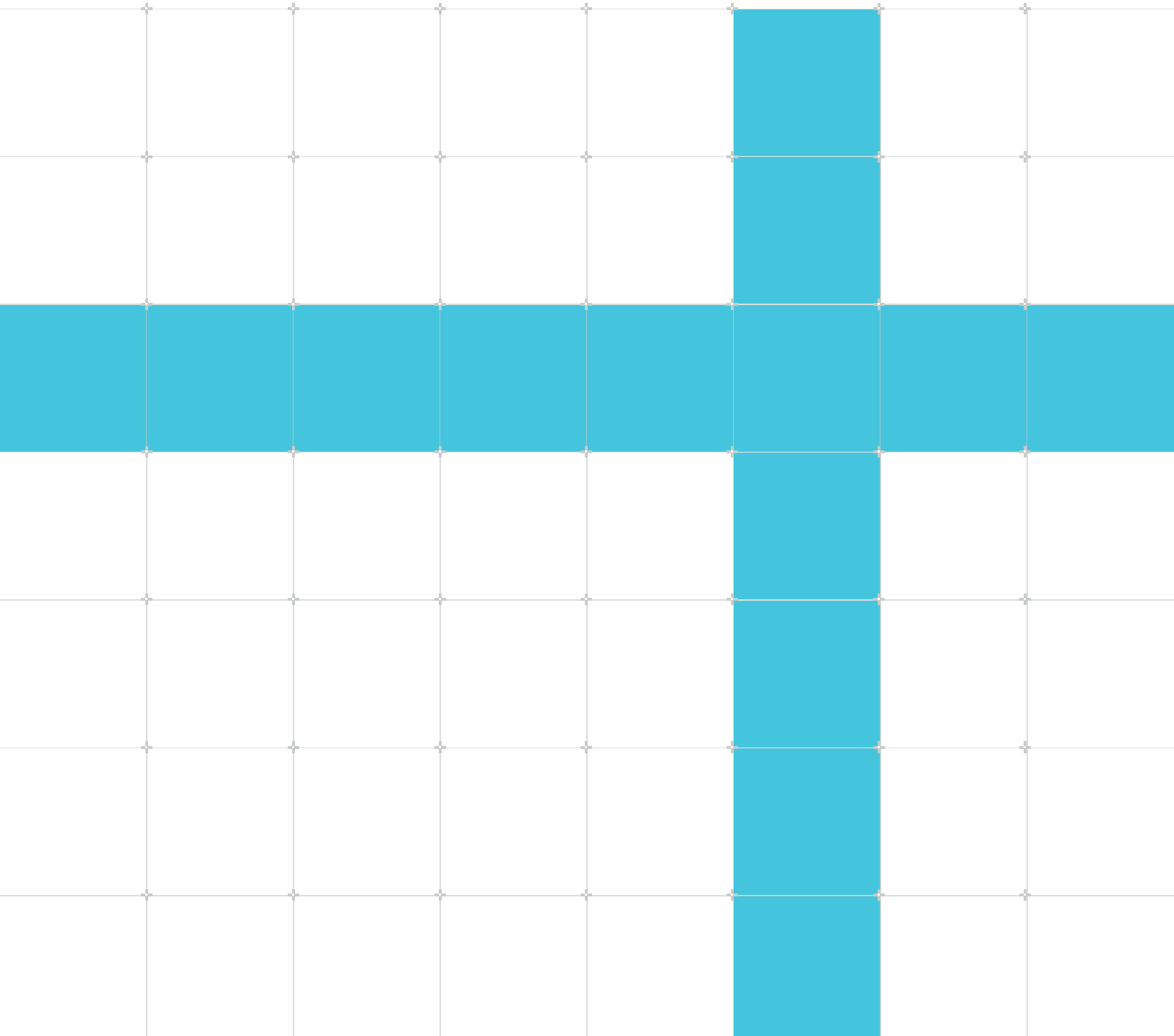
Known issues

Non-Confidential

Copyright © 2020, 2022–2026 Arm Limited (or its affiliates).
All rights reserved.

Issue 01

102105_M.a_01_en



Arm® Architecture Reference Manual for A-profile architecture

Known issues

Copyright © 2020, 2022–2026 Arm Limited (or its affiliates). All rights reserved.

Release information

Document history

Issue	Date	Confidentiality	Change
M.a-01	5 January 2026	Non-Confidential	Known Issues in Arm® Architecture Reference Manual, Issue M.a, as of 5 January 2026
M.a-00	12 December 2025	Non-Confidential	Known Issues in Arm® Architecture Reference Manual, Issue M.a, as of 5 December 2025
L.b-06	3 December 2025	Non-Confidential	Known Issues in Arm® Architecture Reference Manual, Issue L.b, as of 30 November 2025
L.b-05	4 November 2025	Non-Confidential	Known Issues in Arm® Architecture Reference Manual, Issue L.b, as of 3 November 2025
L.b-04	3 October 2025	Non-Confidential	Known Issues in Arm® Architecture Reference Manual, Issue L.b, as of 1 October 2025
L.b-03	3 September 2025	Non-Confidential	Known Issues in Arm® Architecture Reference Manual, Issue L.b, as of 2 September 2025
L.b-02	4 August 2025	Non-Confidential	Known Issues in Arm® Architecture Reference Manual, Issue L.b, as of 4 August 2025
L.b-01	3 July 2025	Non-Confidential	Known Issues in Arm® Architecture Reference Manual, Issue L.b, as of 1 July 2025
L.b-00	30 May 2025	Non-Confidential	Known Issues in Arm® Architecture Reference Manual, Issue L.b, as of 28 May 2025
L.a-05	6 May 2025	Non-Confidential	Known Issues in Arm® Architecture Reference Manual, Issue L.a, as of 6 May 2025
L.a-04	1 April 2025	Non-Confidential	Known Issues in Arm® Architecture Reference Manual, Issue L.a, as of 1 April 2025
L.a-03	6 March 2025	Non-Confidential	Known Issues in Arm® Architecture Reference Manual, Issue L.a, as of 6 March 2025
L.a-02	7 February 2025	Non-Confidential	Known Issues in Arm® Architecture Reference Manual, Issue L.a, as of 7 February 2025

Issue	Date	Confidentiality	Change
L.a-01	7 January 2025	Non-Confidential	Known Issues in Arm® Architecture Reference Manual, Issue L.a, as of 7 January 2025
L.a-00	2 December 2024	Non-Confidential	Known Issues in Arm® Architecture Reference Manual, Issue L.a, as of 2 December 2024
K.a-08	30 November 2024	Non-Confidential	Known Issues in Arm® Architecture Reference Manual, Issue K.a, as of 30 November 2024
J.a-08	4 March 2024	Non-Confidential	Known Issues in Arm® Architecture Reference Manual, Issue J.a, as of 4 March 2024
I.a-06	21 April 2023	Non-Confidential	Known Issues in Arm® Architecture Reference Manual, Issue I.a, as of 31 March 2023
H.a-06	22 July 2022	Non-Confidential	Known Issues in Arm® Architecture Reference Manual, Issue H.a, as of 22 July 2022
G.b-05	31 January 2022	Non-Confidential	Known Issues in Arm® Architecture Reference Manual, Issue G.b, as of 7 January 2022
F.c-04	18 December 2020	Non-Confidential	Known Issues in Arm® Architecture Reference Manual, Issue F.c, as of 18 December 2020

Proprietary Notice

This document is protected by copyright and other related rights and the use or implementation of the information contained in this document may be protected by one or more patents or pending patent applications. No part of this document may be reproduced in any form by any means without the express prior written permission of Arm Limited ("Arm"). No license, express or implied, by estoppel or otherwise to any intellectual property rights is granted by this document unless specifically stated.

Your access to the information in this document is conditional upon your acceptance that you will not use or permit others to use the information for the purposes of determining whether the subject matter of this document infringes any third party patents.

The content of this document is informational only. Any solutions presented herein are subject to changing conditions, information, scope, and data. This document was produced using reasonable efforts based on information available as of the date of issue of this document. The scope of information in this document may exceed that which Arm is required to provide, and such additional information is merely intended to further assist the recipient and does not represent Arm's view of the scope of its obligations. You acknowledge and agree that you possess the necessary expertise in system security and functional safety and that you shall be solely responsible for compliance with all legal, regulatory, safety and security related requirements concerning your products, notwithstanding any information or support that may be provided by

Arm herein. In addition, you are responsible for any applications which are used in conjunction with any Arm technology described in this document, and to minimize risks, adequate design and operating safeguards should be provided for by you.

This document may include technical inaccuracies or typographical errors. THIS DOCUMENT IS PROVIDED "AS IS". ARM PROVIDES NO REPRESENTATIONS AND NO WARRANTIES, EXPRESS, IMPLIED OR STATUTORY, INCLUDING, WITHOUT LIMITATION, THE IMPLIED WARRANTIES OF MERCHANTABILITY, SATISFACTORY QUALITY, NON-INFRINGEMENT OR FITNESS FOR A PARTICULAR PURPOSE WITH RESPECT TO THE DOCUMENT. For the avoidance of doubt, Arm makes no representation with respect to, and has undertaken no analysis to identify or understand the scope and content of, any patents, copyrights, trade secrets, trademarks, or other rights.

TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL ARM BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF ANY USE OF THIS DOCUMENT, EVEN IF ARM HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Reference by Arm to any third party's products or services within this document is not an express or implied approval or endorsement of the use thereof.

This document consists solely of commercial items. You shall be responsible for ensuring that any permitted use, duplication, or disclosure of this document complies fully with any relevant export laws and regulations to assure that this document or any portion thereof is not exported, directly or indirectly, in violation of such export laws. Use of the word "partner" in reference to Arm's customers is not intended to create or refer to any partnership relationship with any other company. Arm may make changes to this document at any time and without notice.

This document may be translated into other languages for convenience, and you agree that if there is any conflict between the English version of this document and any translation, the terms of the English version of this document shall prevail.

The validity, construction and performance of this notice shall be governed by English Law.

The Arm corporate logo and words marked with ® or ™ are registered trademarks or trademarks of Arm Limited (or its affiliates) in the US and/or elsewhere. Please follow Arm's trademark usage guidelines at <https://www.arm.com/company/policies/trademarks>. All rights reserved. Other brands and names mentioned in this document may be the trademarks of their respective owners.

Arm Limited. Company 02557590 registered in England.

110 Fulbourn Road, Cambridge, England CB1 9NJ.

PRE-1121-V1.0

Confidentiality Status

This document is Non-Confidential. The right to use, copy and disclose this document may be subject to license restrictions in accordance with the terms of the agreement entered into by Arm and the party that Arm delivered this document to.

Unrestricted Access is an Arm internal classification.

Product Status

The information in this document is Final, that is for a developed product.

Feedback

Arm welcomes feedback on this product and its documentation. To provide feedback on the product, create a ticket on <https://support.developer.arm.com>

To provide feedback on the document, fill the following survey: <https://developer.arm.com/documentation-feedback-survey>.

Inclusive language commitment

Arm values inclusive communities. Arm recognizes that we and our industry have used language that can be offensive. Arm strives to lead the industry and create change.

We believe that this document contains no offensive language. To report offensive language in this document, email terms@arm.com.

Contents

1. Introduction.....	9
1.1 Conventions.....	9
1.2 Useful resources.....	10
1.3 Other information.....	11
2. Arm Architecture Reference Manual for A-profile architecture.....	12
2.1 D16002.....	12
2.2 D17119.....	13
2.3 R17748.....	13
2.4 D18235.....	14
2.5 D19269.....	15
2.6 C19388.....	16
2.7 D21014.....	18
2.8 D21904.....	20
2.9 C21963.....	20
2.10 C22137.....	22
2.11 E22689.....	23
2.12 C22710.....	25
2.13 D22719.....	25
2.14 C23095.....	28
2.15 D23122.....	31
2.16 D23136.....	33
2.17 R23240.....	34
2.18 D23442.....	36
2.19 R23526.....	38
2.20 C23528.....	39
2.21 R23639.....	39
2.22 C23821.....	40
2.23 R23822.....	41
2.24 C23965.....	42
2.25 C24066.....	44
2.26 C24074.....	45

2.27 C24078.....	47
2.28 D24095.....	50
2.29 C24109.....	51
2.30 D24150.....	53
2.31 D24156.....	55
2.32 D24241.....	55
2.33 D24315.....	56
2.34 R24327.....	56
2.35 D24361.....	57
2.36 D24371.....	58
2.37 R24482.....	59
2.38 C24523.....	61
2.39 R24537.....	63
2.40 C24538.....	64
2.41 D24547.....	65
2.42 D24553.....	66
2.43 D24555.....	66
2.44 R24557.....	67
2.45 C24563.....	68
2.46 D24566.....	69
2.47 D24595.....	69
2.48 D24642.....	70
2.49 D24658.....	70
2.50 D24660.....	70
2.51 R24661.....	73
2.52 D24669.....	77
2.53 C24673.....	78
2.54 R24675.....	78
2.55 R24687.....	82
2.56 D24695.....	83
2.57 D24696.....	84
2.58 D24722.....	84
2.59 D24726.....	85
2.60 D24779.....	86
2.61 D24784.....	87
2.62 C24810.....	87

2.63 C24818.....	88
2.64 D24835.....	88
2.65 E24841.....	89
2.66 R24849.....	90
2.67 C24856.....	91
2.68 R24862.....	91
2.69 D24867.....	92
2.70 D24887.....	92
2.71 C24897.....	93
2.72 D24952.....	95
2.73 D24957.....	95
2.74 D24960.....	95
2.75 C25027.....	97
2.76 C25036.....	97
2.77 R25037.....	98
2.78 D25054.....	100
2.79 D25060.....	100
3. Arm Memory System Resource Partitioning and Monitoring (MPAM) System Component Specification.....	103
4. Arm Reliability, Availability, and Serviceability (RAS) System Architecture for A-profile architecture.....	104
4.1 D24969.....	104

1. Introduction

1.1 Conventions

The following subsections describe conventions used in Arm documents.

Glossary

The Arm Glossary is a list of terms used in Arm documentation, together with definitions for those terms. The Arm Glossary does not contain terms that are industry standard unless the Arm meaning differs from the generally accepted meaning.

See the Arm Glossary for more information: developer.arm.com/glossary.

Typographic conventions

Arm documentation uses typographical conventions to convey specific meaning.

Convention	Use
italic	Citations.
bold	Interface elements, such as menu names. Terms in descriptive lists, where appropriate.
monospace	Text that you can enter at the keyboard, such as commands, file and program names, and source code.
monospace <u>underline</u>	A permitted abbreviation for a command or option. You can enter the underlined text instead of the full command or option name.
<and>	Encloses replaceable terms for assembler syntax where they appear in code or code fragments. For example: MRC p15, 0, <Rd>, <CRn>, <CRm>, <Opcode_2>
SMALL CAPITALS	Terms that have specific technical meanings as defined in the <i>Arm® Glossary</i> . For example, IMPLEMENTATION DEFINED , IMPLEMENTATION SPECIFIC , UNKNOWN , and UNPREDICTABLE .



Caution

We recommend the following. If you do not follow these recommendations your system might not work.



Your system requires the following. If you do not follow these requirements your system will not work.



You are at risk of causing permanent damage to your system or your equipment, or harming yourself.



This information is important and needs your attention.



A useful tip that might make it easier, better or faster to perform a task.



A reminder of something important that relates to the information you are reading.

1.2 Useful resources

This document contains information that is specific to this product. See the following resources for other useful information.

Arm documents are available on developer.arm.com/documentation.

Confidential documents are only available to licensees, when logged in. Each document link in the following tables provides direct access to the online version of the document.

Arm product resources	Document ID	Confidentiality
Arm® Architecture Reference Manual for A-profile architecture, Issue M.a.a	DDI 0487M.a.a	Non-Confidential
Arm® Memory System Resource Partitioning and Monitoring (MPAM) System Component Specification, Issue B.a	IHI 0100A.b	Non-Confidential
Arm® Reliability, Availability, and Serviceability (RAS) System Architecture, Issue A.b	IHI 0100A.b	Non-Confidential

1.3 Other information

See the Arm website for other relevant information.

- [Arm® Developer](#).
- [Arm® Documentation](#).
- [Technical Support](#).
- [Arm® Glossary](#).

2. Arm Architecture Reference Manual for A-profile architecture

This section records known issues in the Arm Architecture Reference Manual for A-profile architecture (DDI 0487), Issue M.a.

Key

- C = Clarification.
- D = Defect.
- R = Relaxation.
- E = Enhancement.

2.1 D16002

In section **D24.2.45 “ESR_EL1, Exception Syndrome Register (EL1)”**, under the heading “ISS encoding for an exception from a Data Abort”, the description of the ‘SET, bits [12:11]’ field that reads:

When `IsFeatureImplemented(FEAT_RAS) && ((DFSC == 0b010000 || DFSC IN {0b01001x}) || DFSC IN {0b0101xx})`:

SET, bits [12:11]

Synchronous Error Type. Used when a synchronous External abort, not on a Translation table walk or hardware update of the Translation table, generated the Data Abort. Describes the PE error state after taking the Data Abort exception.

is changed to read:

When `IsFeatureImplemented(FEAT_RAS) && DFSC == 0b010000`:

SET, bits [12:11]

Synchronous Error Type. Used when a synchronous External abort, not on a Translation table walk or hardware update of the Translation table, generated the Data Abort. Describes the PE error state after taking the Data Abort exception.

Equivalent changes are made in the following sections:

- **D24.2.46 “ESR_EL2, Exception Syndrome Register (EL2)”**.
- **D24.2.47 “ESR_EL3, Exception Syndrome Register (EL3)”**.

2.2 D17119

In sections **F3.1.10.2 “Advanced SIMD two registers and shift amount”** and **F4.1.22.2 “Advanced SIMD two registers and shift amount”**, the following constraints are added to VMOVL:

- 'L' must be 'O'.
- 'imm3H' cannot be '000'.

2.3 R17748

In section **H2.2.6 “Imprecise entry to Debug state”**, the text that reads:

■ An External Debug Request debug event must be pending before writing 1 to this bit.

is changed to read:

■ If an External Debug Request debug event is not pending when 1 is written to this bit, and the write is not ignored by the PE, then the resulting behavior of the PE is **UNPREDICTABLE**.

Additionally, in the same section, the text that reads:

■ The PE ignores writes to this bit in all the following cases:

- `ExternallInvasiveDebugEnabled() == FALSE`.
- FEAT_RME is not implemented and `ExternalSecureInvasiveDebugEnabled() == FALSE`, and either:
 - EL3 is not implemented and the implemented Security state is Secure state.
 - EL3 is implemented.
- FEAT_RME is implemented and `ExternalRootInvasiveDebugEnabled() == FALSE`.

is changed to read:

■ The PE ignores writes to `EDRCR.CBRRQ` if any of the following apply:

- `ExternallInvasiveDebugEnabled() == FALSE`.
- FEAT_RME is not implemented and `ExternalSecureInvasiveDebugEnabled() == FALSE`, and either:
 - EL3 is not implemented and the implemented Security state is Secure state.
 - EL3 is implemented.
- FEAT_RME is implemented and `ExternalRootInvasiveDebugEnabled() == FALSE`.
- Any **IMPLEMENTATION DEFINED** reason.

In the same section, the following guidance is added:

Arm recommends that the PE ignores writes to this bit if an External Debug Request debug event is not pending.

If forcing this PE to enter Debug state imprecisely could have an adverse effect on the wider system, then Arm recommends that a write to EDRCCR.CBRRQ is ignored if the highest level of debug authentication is not enabled at the system level.

2.4 D18235

In section **D24.3.6 “DBGDTR_EL0, Debug Data Transfer Register, half-duplex”**, the text that reads:

HighWord, bits [63:32]

Writes to this register set DTRRX to the value in this field and do not change RXfull.

...

LowWord, bits [31:0]

Writes to this register set DTRTX to the value in this field and set TXfull to 1.

is changed to read:

HighWord, bits [63:32]

If TXfull is 1, then writes to this register set DTRRX to an **UNKNOWN** value. Otherwise, writes to this register set DTRRX to the value in this field. Writes to this register do not change RXfull.

...

LowWord, bits [31:0]

If TXfull is 1, then writes to this register set DTRTX to an **UNKNOWN** value. Otherwise, writes to this register set DTRTX to the value in this field and set TXfull to 1.

In section **H4.5.2 “DCC accesses in Non-debug state”**, the note that reads:

These ordering rules mean that software:

- Must not read DBGDTRRX without first checking DCCSR.RXfull or if the previously-read value of DCCSR.RXfull is 0. It is not sufficient to read both registers and then later decide whether to discard the read value, as there might be an intervening write from the external debug interface.
- Must not write DBGDTRTX without first checking DCCSR.TXfull or if the previously-read value of DCCSR.TXfull is 1. The write to DBGDTRTX overwrites the value in DTRTX, and the external debugger might or might not have read this value.

- Must ensure there is an explicit Context Synchronization event following a DTR access, even if not immediately returning to read DCCSR again. This synchronization operation can be an exception return.

is changed to read:

These ordering rules mean that software:

- Must not read DBGDTRRX or DBGDTR_ELO without first checking DCCSR.RXfull or if the previously-read value of DCCSR.RXfull is 0. It is not sufficient to read both registers and then later decide whether to discard the read value, as there might be an intervening write from the external debug interface.
- Must not write DBGDTRTX or DBGDTR_ELO without first checking DCCSR.TXfull or if the previously-read value of DCCSR.TXfull is 1. The write overwrites the value in DTRTX, and the external debugger might or might not have read this value.
- Must ensure there is an explicit Context Synchronization event following a DTR access, even if not immediately returning to read DCCSR again. This synchronization operation might be an exception return.

2.5 D19269

In section **D24.7.5 “PMBSR_EL1, Profiling Buffer Status/syndrome Register (EL1)”**, in the description of field ‘DL, bit [19]’, the text that reads:

DL, bit [19]

Partial record lost. Following a buffer management event other than an asynchronous External abort, indicates whether the last record written to the Profiling Buffer is complete.

DL	Meaning
0b0	PMBPTR_EL1 points to the first byte after the last complete record written to the Profiling Buffer.
0b1	Part of a record was lost because of a buffer management event or synchronous External abort. PMBPTR_EL1 might not point to the first byte after the last complete record written to the buffer, and so restarting collection might result in a data record stream that software cannot parse.

When the buffer management event was because of an asynchronous External abort, this bit is set to 1 and software must not assume that any valid data has been written to the Profiling Buffer.

This bit is **RES0** if the PE never sets this bit as a result of a buffer management event caused by an asynchronous External abort.

is changed to read:

DL, bit [19]

Partial record lost. Following a buffer management event, indicates whether the last record written to the Profiling Buffer is complete. PMBSR_EL1.DL is also set to 1 when an asynchronous External abort is reported to the SPU.

DL	Meaning
0b0	PMBPTR_EL1 points to the first byte after the last complete record written to the Profiling Buffer by the SPU, and no asynchronous External abort has been reported to the SPU.
0b1	Part of a record was lost because of a Profiling Buffer management event, or an asynchronous External abort has been reported to the SPU. PMBPTR_EL1 might not be the address of the first byte after the last complete sample record written to the Profiling Buffer by the SPU.

When the SPU sets this bit to 1, software must not assume that there is any valid data between the end of the last complete record and PMBPTR_EL1. If software restarts profiling from the saved PMBPTR_EL1 value, then this might result in buffer contents that software cannot parse. If PMBSR_EL1.EA is also set to 1 by the SPU, then software must not assume that any valid data has been written to the Profiling Buffer.

This bit is **RES0** if there are no circumstances when the PE would ever set this bit to 1 as a result of a Profiling Buffer management event.

2.6 C19388

In section **A2.2.2 “The Armv8.1 architecture extension”**, the text that reads:

FEAT_HAFDBS, Hardware management of the Access flag and dirty state

In Armv8.0, all updates to the translation tables are performed by software. From Armv8.1, for the VMSAv8-64 translation regimes only, hardware can perform updates to the translation tables in two contexts:

- Hardware management of the Access flag.
- Hardware management of dirty state, with updates to a dirty state in the translation tables.

The dirty state is introduced in Armv8.1.

Hardware management of dirty state can be enabled only when hardware management of the Access flag is also enabled.

This feature is supported in AArch64 state only.

FEAT_HAFDBS is OPTIONAL from Armv8.0.

The following field identifies the presence of FEAT_HAFDBS:

- ID_AA64MMFR1_EL1.HAFDBS.

For more information, see:

- Hardware management of the Access flag.

- Hardware management of the dirty state.

is changed to read:

FEAT_HAF, Hardware management of the Access flag.

Hardware can perform updates to the translation tables to manage the Access flag.

FEAT_HAF is optional from Armv8.0.

The following field identifies the presence of FEAT_HAF:

- ID_AA64MMFR1_EL1.HAFDBS.

For more information, see 'Hardware management of the Access flag'.

FEAT_HAFDBS, Hardware management of dirty state

Hardware can perform updates to dirty state in the translation tables.

FEAT_HAFDBS is optional from Armv8.0.

If FEAT_HAFDBS is implemented, then FEAT_HAF is implemented.

The following field identifies the presence of FEAT_HAFDBS:

- ID_AA64MMFR1_EL1.HAFDBS.

For more information, see 'Hardware management of the dirty state'.

In section **D24.2.89 "ID_AA64MMFR1_EL1, AArch64 Memory Model Feature Register 1"**, in the field description of 'HAFDBS, bits [3:0]', the text that reads:

FEAT_HAFDBS implements the functionality identified by the values 0b0001 and 0b0010.

is changed to read:

FEAT_HAF implements the functionality identified by the value 0b0001.

FEAT_HAFDBS implements the functionality identified by the value 0b0010.

In section **D24.2.194 "TCR_EL1, Translation Control Register (EL1)"**, in the field description of 'HA, bit [39]', the text that reads:

When FEAT_HAFDBS is implemented:

is changed to read:

When FEAT_HAF is implemented:

Equivalent changes are made in the 'HA' fields in the following sections:

- **D24.2.195 "TCR_EL2, Translation Control Register (EL2)"**.

- **D24.2.196 “TCR_EL3, Translation Control Register (EL3)”.**
- **D24.2.224 “VTCR_EL2, Virtualization Translation Control Register”.**

References to Access flag updates mentioning FEAT_HAFDBS are changed to read FEAT_HAF, and corresponding updates are made to the pseudocode description.

2.7 D21014

In section **D6.5.4, “Faults”**, in rule R_{FSPBK} , the text that reads:

If a write by the Trace Buffer Unit generates an External abort on a translation table walk or translation table update, it is **IMPLEMENTATION DEFINED** whether TRBSR_EL1.EA is set to 1 or unchanged.

is changed to read:

If a write by the Trace Buffer Unit generates an External abort on a translation table walk, translation table update, or GPT walk, then it is **IMPLEMENTATION DEFINED** whether TRBSR_EL1.EA is set to 1 or unchanged.

In section **H9.4.30, “TRBSR_EL1, Trace Buffer Status/syndrome Register”**, the text that reads:

EA, bit [18]

From Armv9.3:

Reserved, **RES0**.

When the PE sets this bit as the result of an External Abort:

External Abort.

0b0	An External Abort has not been asserted.
0b1	An External Abort has been asserted and detected by the Trace Buffer Unit.

The reset behavior of this field is:

- On a Cold reset, this field resets to an architecturally **UNKNOWN** value.

Otherwise:

Reserved, **RES0**.

is changed to read:

EA, bit [18]

When the PE sets this bit as the result of an External abort:

External Abort.

0b0	An External Abort has not been asserted.
0b1	An External Abort has been asserted and detected by the Trace Buffer Unit.

It is **IMPLEMENTATION DEFINED** whether this field is set to 1 or is unchanged by the PE when a write by the Trace Buffer Unit generates an External abort on a translation table walk, translation table update, or GPT walk that is reported as an MMU fault. From Armv9.3, this field is not set to 1 by the PE for any other trace buffer management event.

The reset behavior of this field is:

- On a Cold reset, this field resets to an architecturally **UNKNOWN** value.

Otherwise:

Reserved, **RES0**.

In section **D17.8.4 “External aborts”**, the text that reads:

If a write to the Profiling Buffer generates an External abort on a translation table walk or translation table update, it is **IMPLEMENTATION DEFINED** whether PMBSR_EL1.EA is set to 1 or unchanged.

is changed to read:

If a write to the Profiling Buffer generates an External abort on a translation table walk, translation table update, or GPT walk, then it is **IMPLEMENTATION DEFINED** whether PMBSR_EL1.EA is set to 1 or unchanged.

In section **D24.7.5 “PMBSR_EL1, Profiling Buffer Status/syndrome Register (EL1)”**, the text that reads:

EA, bit [18]

External abort.

0b0	An External abort has not been asserted.
0b1	An External abort has been asserted and detected by the Statistical Profiling Unit.

This bit is **RES0** if the PE never sets this bit as the result of an External abort.

The reset behavior of this field is:

- On a Warm reset, this field resets to an architecturally **UNKNOWN** value.

is changed to read:

EA, bit [18]

When the PE sets this bit as the result of an External abort:

External abort.

0b0	An External abort has not been asserted.
0b1	An External abort has been asserted and detected by the Statistical Profiling Unit.

It is **IMPLEMENTATION DEFINED** whether this field is set to 1 or is unchanged by the PE when a write by the Statistical Profiling Unit generates an External abort on a translation table walk, translation table update, or GPT walk that is reported as an MMU fault. When FEAT_SPEv1p3 is implemented, this field is not set to 1 for any other Profiling Buffer management event.

The reset behavior of this field is:

- On a Warm reset, this field resets to an architecturally **UNKNOWN** value.

Otherwise:

Reserved, **RES0**.

2.8 D21904

In section **D24.2.85 “ID_AA64ISAR1_EL1, AArch64 Instruction Set Attribute Register 1”**, the description of value 0b0010 in field LRCPC that reads:

As 0b0001, and the LDAPR (unscaled immediate) and STLR (unscaled immediate) instructions are implemented.

is changed to read:

As 0b0001, and the LDAPUR (unscaled), LDAPURB (unscaled), LDAPURSB (unscaled), LDAPURH (unscaled), LDAPURSH (unscaled), LDAPURSW (unscaled), STLUR (unscaled), STLURB (unscaled) and STLURH (unscaled) instructions are implemented.

2.9 C21963

In section **D24.2.93 “ID_AA64PFR0_EL1, AArch64 Processor Feature Register 0”**, the text that reads:

FEAT_RAS implements the functionality identified by the value 0b0001.

FEAT_RASv1p1 and FEAT_DoubleFault implement the functionality identified by the value 0b0010.

FEAT_RASv2 implements the functionality identified by the value 0b0011.

In Armv8.0 and Armv8.1, the permitted values are 0b0000 and 0b0001.

From Armv8.2, the value 0b0000 is not permitted.

From Armv8.4, if FEAT_DoubleFault is implemented or ERRIDR_EL1.NUM is nonzero, the value 0b0001 is not permitted.

Note:

When the value of this field is 0b0001, ID_AA64PFR1_EL1.RAS_frac indicates whether FEAT_RASv1p1 is implemented.

From Armv8.9, if ERRIDR_EL1.NUM is nonzero, the value 0b0010 is not permitted.

is changed to read:

FEAT_RAS implements the functionality identified by the value 0b0001.

FEAT_DoubleFault implements the functionality identified by the value 0b0010.

When ERRIDR_EL1.NUM is zero:

- If FEAT_DoubleFault is not implemented, then this field is permitted to have the value 0b0001, regardless of whether FEAT_RASv1p1 or FEAT_RASv2 are implemented.
- If FEAT_DoubleFault is implemented, then this field is permitted to have the value 0b0010, regardless of whether FEAT_RASv2 is implemented.

Otherwise:

- FEAT_RASv1p1 implements the functionality identified by the value 0b0010.
- FEAT_RASv2 implements the functionality identified by the value 0b0011.

In Armv8.0 and Armv8.1, the permitted values are 0b0000 and 0b0001.

From Armv8.2, the value 0b0000 is not permitted.

From Armv8.4, if FEAT_DoubleFault is implemented or ERRIDR_EL1.NUM is nonzero, then the value 0b0001 is not permitted.

Note

When the value of this field is 0b0001, ID_AA64PFR1_EL1.RAS_frac indicates whether FEAT_RASv1p1 is implemented.

From Armv8.9, if ERRIDR_EL1.NUM is nonzero, then the value 0b0010 is not permitted.

The equivalent changes are made in the following sections:

- **D24.2.114 “ID_PFR0_EL1, AArch32 Processor Feature Register 0”.**
- **G8.2.99 “ID_PFR0, Processor Feature Register 0”.**

Additionally, in section **D24.2.94 “ID_AA64PFR1_EL1, AArch64 Processor Feature Register 1”**, the text that reads:

All other values are reserved.

FEAT_RAS implements the functionality identified by the value 0b0000.

FEAT_RASv1p1 implements the functionality identified by the value 0b0001.

This field is valid only if ID_AA64PFR0_EL1.RAS == 0b0001.

is changed to read:

All other values are reserved.

FEAT_RAS implements the functionality identified by the value 0b0000.

When ERRIDR_EL1.NUM is zero, this field is permitted to have the value 0b0000, regardless of whether FEAT_RASv1p1 is implemented.

Otherwise, FEAT_RASv1p1 implements the functionality identified by the value 0b0001.

This field is valid only if ID_AA64PFR0_EL1.RAS == 0b0001.

The equivalent changes are made in the following sections:

- **D24.2.116 “ID_PFR2_EL1, AArch32 Processor Feature Register 2”.**
- **G8.2.101 “ID_PFR2, Processor Feature Register 2”.**

2.10 C22137

In the **Glossary**, in the definition of **RES0**, the text that reads:

A reserved bit. Used for fields in register descriptions, and for fields in architecturally-defined data structures that are held in memory, for example in translation table descriptors.

...

A bit that is **RES0** in a context is reserved for possible future use in that context. To preserve forward compatibility, software:

- Must not rely on the bit reading as 0.
- Must use an SBZP policy to write to the bit.

is changed to read:

A reserved bit. Used for fields in register descriptions, and for fields in architecturally-defined data structures that are held in memory, for example in translation table descriptors.

...

A bit that is **RES0** in a context is reserved for possible future use in that context. To preserve forward compatibility, software:

- Must not rely on the bit reading as 0.
- Must use an SBZP policy to write to the bit.

The definition of a **RES0** field in an architecturally-defined data structure that is held in memory is:

- A read of a **RES0** bit returns the value last written.
- The value of a **RES0** bit has no effect on the operation of the PE or any other hardware implementing the architecture such as the GIC or the SMMU, other than determining the value read back from that bit, unless this Manual explicitly defines additional properties for the bit.
- For an implicit memory write effect to the field by a PE or for a write to the field by any other hardware implementing the architecture such as the GIC or the SMMU, each bit defined as **RES0** is set to 0.
- For a write to the data structure by a PE, or any other hardware implementing the architecture such as the GIC or the SMMU, if the write is specified to only update specific fields, the hardware preserves the value of other **RES0** bits in the same data structure. This applies, for example, to Hardware updates to the translation tables.

A similar clarification is added to the **RES1** definition.

2.11 E22689

In **D24.2.80 “ID_AA64DFR0_EL1, AArch64 Debug Feature Register 0”**, in the field description of ‘BRPs, bits [15:12]’, the text that reads:

Number of breakpoints, minus 1. The value of this field is an **IMPLEMENTATION DEFINED** choice of:

0b0001...0b1111	The number of breakpoints, minus 1.
-----------------	-------------------------------------

If FEAT_Debugv8p9 is implemented and 16 or more breakpoints are implemented, then this field reads as 0b1111 and ID_AA64DFR1_EL1.BRPs indicates the number of breakpoints.

is changed to read:

Index of the highest numbered context-aware breakpoint. Identifies which of the implemented breakpoints are context-aware breakpoints.

The value of this field is an **IMPLEMENTATION DEFINED** choice of:

0b0001...0b1110	Index of the highest numbered context-aware breakpoint.
0b1111	If ID_AA64DFR0_EL1.CTX_CMPs and ID_AA64DFR1_EL1.CTX_CMPs indicate that 16 or fewer breakpoints are context-aware, then the index of the highest context-aware breakpoint is 15. Otherwise, the context-aware breakpoints are the lowest numbered breakpoints.

If ID_AA64DFR1_EL1.BRPs is zero, then this is also the number of implemented breakpoints, minus 1, meaning the context-aware breakpoints are the highest numbered breakpoints.

The value of this field is always less than the number of implemented breakpoints.

In the same section, in the field description of 'CTX_CMPs, bits [31:28]', the text that reads:

Number of context-aware breakpoints, minus 1.

The value of this field is an **IMPLEMENTATION DEFINED** choice of:

0b0000...0b1111	The number of context-aware breakpoints, minus 1.
-----------------	---

The value of this field is never greater than ID_AA64DFR0_EL1.BRPs. If FEAT_Debugv8p9 is implemented and 16 or more context-aware breakpoints are implemented, then this field reads as 0b1111 and ID_AA64DFR1_EL1.CTX_CMPs indicates the number of context-aware breakpoints.

is changed to read:

Number of context-aware breakpoints, minus 1.

The value of this field is an **IMPLEMENTATION DEFINED** choice of:

0b0000...0b1110	The number of context-aware breakpoints, minus 1.
0b1111	If ID_AA64DFR1_EL1.CTX_CMPs is zero, then 16 context-aware breakpoints are implemented. Otherwise, 16 or more context-aware breakpoints are implemented and ID_AA64DFR1_EL1.CTX_CMPs indicates how many.

Values greater than ID_AA64DFR0_EL1.BRPs are not permitted.

In **D24.2.81 "ID_AA64DFR1_EL1, AArch64 Debug Feature Register 1"**, in the field description of 'BRPs, bits [15:8]', the following text is added:

If more than 16 breakpoints are implemented, or the index of the highest numbered context-aware breakpoint is not the number of breakpoints minus 1, then the value 0x00 is not permitted.

Nonzero values less than ID_AA64DFR0_EL1.BRPs are not permitted.

In the same section, in the field description of 'CTX_CMPs, bits [31:24]', the text that reads:

The value of this field is never greater than ID_AA64DFR1_EL1.BRPs.

is changed to read:

If 16 or fewer context-aware breakpoints are implemented, then the value of this field is either 0x00 or the same as ID_AA64DFR0_EL1.CTX_CMPs. If more than 16 context-aware breakpoints are implemented, then the value 0x00 is not permitted.

Values greater than ID_AA64DFR1_EL1.BRPs are not permitted.

2.12 C22710

In section **D24.2.45 “ESR_EL1, Exception Syndrome Register (EL1)”**, the text that reads:

ISS encoding for an exception from Branch Target Identification instruction

is changed to read:

ISS encoding for a Branch Target Exception

The same change is made in the following sections:

- **D24.2.46 “ESR_EL2, Exception Syndrome Register (EL2)”**.
- **D24.2.47 “ESR_EL3, Exception Syndrome Register (EL3)”**.

2.13 D22719

In **C5.5.92 “TLBIP RIPAS2E1, TLBIP RIPAS2E1NXS, TLB Range Invalidate by Intermediate Physical Address, Stage 2, EL1”**, under the heading ‘TTL, bits [38:37]’, the following text is removed:

If FEAT_LPA2 is not implemented, when using a 16KB translation granule, this value is reserved and hardware should treat this field as 0b00.

The equivalent change is applied to all other TLBIP instructions that apply to a range of addresses.

In **C5.5.86 “TLBIP IPAS2E1, TLBIP IPAS2E1NXS, TLB Invalidate Pair by Intermediate Physical Address, Stage 2, EL1”**, under heading ‘TTL, bits [47:44]’, the text that reads:

- 0b01xx The entry comes from a 4KB translation granule. The level of walk for the leaf level 0bxx is encoded as:
 - 0b00 : If FEAT_LPA2 is implemented, level 0. Otherwise, treat as if TTL[3:2] is 0b00.
 - ...
- 0b10xx The entry comes from a 16KB translation granule. The level of walk for the leaf level 0bxx is encoded as:
 - 0b00 : Reserved. Treat as if TTL[3:2] is 0b00.
 - 0b01 : If FEAT_LPA2 is implemented, level 1. Otherwise, treat as if TTL[3:2] is 0b00.
 - ...

is changed to read:

- 0b01xx The entry comes from a 4KB translation granule. The level of walk for the leaf level 0bxx is encoded as:
 - 0b00 : Level 0.
 - ...

- 0b10xx The entry comes from a 16KB translation granule. The level of walk for the leaf level 0bxx is encoded as:
 - 0b00 : Reserved. Treat as if TTL[3:2] is 0b00.
 - 0b01 : Level 1.
 - ...

The equivalent change is applied to all other TLBIP instructions that do not apply to a range of addresses.

In **C5.5.59 “TLBI VAE1, TLBI VAE1NXS, TLB Invalidate by VA, EL1”**, under the heading ‘Purpose’, the text that reads:

- The entry is one of the following:
 - A 64-bit stage 1 translation table entry.
 - If FEAT_D128 is implemented, a 128-bit stage 1 translation table entry, if TTL[3:2] is 0b00.
- The entry would be used to translate the specified VA, and one of the following applies:
 - The entry is from a level of lookup above the final level and matches the specified ASID.
 - The entry is a global entry from the final level of lookup.
 - The entry is a non-global entry from the final level of lookup that matches the specified ASID.

is changed to read:

- The entry would be used to translate the specified VA, and one of the following applies:
 - The entry is a Table entry and all the following apply:
 - The entry matches the specified ASID.
 - If FEAT_TTL is implemented, then one of the following applies:
 - TTL[3:2] is 0b00.
 - The entry is in VMSAv8-32 LPAE or VMSAv8-64 format and leads to a Page or Block entry translating the specified VA that matches the granule and level specified in TTL.
- The entry is a Page or Block entry and all of the following apply:
 - For a non-global entry, the entry matches the specified ASID.
 - If FEAT_TTL is implemented, then one of the following applies:
 - TTL[3:2] is 0b00.
 - The entry is in VMSAv8-32 LPAE or VMSAv8-64 format and matches the granule and level specified in TTL.

The equivalent changes are applied to other TLBI instructions that have a TTL hint.

In **C5.5.128 “TLBIP VAE1, TLBIP VAE1NXS, TLB Invalidate Pair by VA, EL1”**, under the heading ‘Purpose’, the text that reads:

- The entry is one of the following:
 - A 128-bit stage 1 translation table entry.
 - A 64-bit stage 1 translation table entry, if TTL[3:2] is 0b00.
- The entry would be used to translate the specified VA, and one of the following applies:
 - The entry is from a level of lookup above the final level and matches the specified ASID.
 - The entry is a global entry from the final level of lookup.
 - The entry is a non-global entry from the final level of lookup that matches the specified ASID.

is changed to read:

- The entry would be used to translate the specified VA, and one of the following applies:
 - The entry is a Table entry and all the following apply:
 - The entry matches the specified ASID.
 - If FEAT_TTL is implemented, then one of the following applies:
 - TTL[3:2] is 0b00.
 - The entry is in VMSAv9-128 format and leads to a Page or Block entry translating the specified VA that matches the granule and level specified in TTL.
- The entry is a Page or Block entry and all of the following apply:
 - For a non-global entry, the entry matches the specified ASID.
 - If FEAT_TTL is implemented, then one of the following applies:
 - TTL[3:2] is 0b00.
 - The entry is in VMSAv9-128 format and matches the granule and level specified in TTL.

The equivalent changes are applied to other TLBIP instructions that have a TTL hint.

In **D8.17.5.3 “Translation table level hint”**, the following new information statement is added:

For non-final levels of walk, a translation using VMSAv9-128 descriptors resolves a different number of IA bits from a translation using VMSAv8-64 descriptors. Because of this mismatch, the TTL hint applies differently between TLBI and TLBIP instructions:

- For TLBI instructions, the TTL hint applies for 64-bit descriptors, including VMSAv8-64 and VMSAv8-32LPAE descriptors.
- For TLBIP instructions, the TTL hint applies for 128-bit descriptors.

2.14 C23095

In section **D24.2.62 “HCR_EL2, Hypervisor Configuration Register”** in field ‘E2H, bit [34]’, the text that reads:

When FEAT_E2H0 is not implemented, this field is **RES1** and behaves as if it is 1 for all purposes other than a direct read.

is changed to read:

When FEAT_E2H0 is not implemented, regardless of whether EL2 is enabled in the current Security state, this field is **RES1** and behaves as if it is 1 for all purposes other than a direct read.

In the same register, in field ‘TGE, bit[27]’, the text that reads:

When EL2 is not enabled in the current Security state, this control has no effect on execution at ELO.

is changed to read:

When EL2 is not enabled in the current Security state, the Effective value of this field is 0 and this field has no effect on execution at ELO.

In section **D24.2.176 “SCTLR_EL2, System Control Register (EL2)”**, in field ‘TIDCP, bit[63]’, the text that reads:

When FEAT_TIDCP1 is implemented and ELIsInHost(EL2):

...

If HCR_EL2.TGE == 0b0, the field is IGNORED for all purposes other than direct reads and writes of the register.

...

Otherwise:

Reserved, **RES0**.

is changed to read:

When FEAT_TIDCP1 is implemented and the Effective value of HCR_EL2.E2H is 1:

...

If the Effective value of HCR_EL2.TGE is 0, the field is IGNORED for all purposes other than direct reads and writes of the register.

...

Otherwise:

Reserved, **RES0**.

Equivalent changes are made in fields 'EnTP2, bit[60]', 'TCSO0, bit[58]', 'EPAN, bit[57]', 'EnAS0, bit[55]', 'EnASR, bit[54]', 'TME0, bit[52]', 'TMT0, bit[50]', 'TWEDEL, bits[49:46]', 'TWEDEn, bit[45]', 'ATA0, bit[42]', 'TCF0, bits[39:38]', 'BT0, bit[35]', 'MSCEn, bit[33]', 'CMOW, bit[32]', 'LSMAOE, bit[29]', 'nTLSMD, bit[28]', 'UCI, bit[26]', 'EOE, bit[24]', 'SPAN, bit[23]', 'nTWE, bit[18]', 'nTWI, bit[16]', 'UCT, bit[15]', 'DZE, bit[14]', 'EnRCTX, bit[10]', 'SED, bit[8]', 'ITD, bit[7]', and 'SA0, bit[4]'.

In the same register, field 'EnFPM, bit[34]' with text that reads:

When FEAT_FPMR is implemented and ELIsInHost(EL0):

...

Otherwise:

Reserved, **RES0**.

is changed to read:

When FEAT_FPMR is implemented and the Effective value of HCR_EL2.E2H is 1:

...

Otherwise:

Reserved, **RES0**.

In the same register, in field 'TSCXT, bit[20]', the text that reads:

When (FEAT_CSV2_2 is implemented or FEAT_CSV2_1p2 is implemented) and ELIsInHost(EL2):

...

When FEAT_CSV2_2 is not implemented, FEAT_CSV2_1p2 is not implemented, and ELIsInHost(EL0):

Reserved, **RES1**.

Otherwise:

Reserved, **RES0**.

is changed to read:

When (FEAT_CSV2_2 is implemented or FEAT_CSV2_1p2 is implemented) and the Effective value of HCR_EL2.E2H is 1:

...

When FEAT_CSV2_2 is not implemented, FEAT_CSV2_1p2 is not implemented, and the Effective value of HCR_EL2.E2H is 1:

Reserved, **RES1**.

Otherwise:

Reserved, **RES0**.

In section **D24.2.171 “SCTLR2_EL2, System Control Register (EL2)”**, in field ‘CPTM0, bit[12]’, the text that reads:

When FEAT_CPA2 is implemented and ELIsInHost(EL2):

...

If HCR_EL2.TGE == 0b0, the field is IGNORED for all purposes other than direct reads and writes of the register.

...

Otherwise:

Reserved, **RES0**.

is changed to read:

When FEAT_CPA2 is implemented and the Effective value of HCR_EL2.E2H is 1:

...

If the Effective value of HCR_EL2.TGE is 0, the field is IGNORED for all purposes other than direct reads and writes of the register.

...

Otherwise:

Reserved, **RES0**.

Equivalent changes are made in fields ‘CPTA0, bit[10]’, and ‘EnPACM, bit[8]’.

In section **D24.10.2 “CNTHCTL_EL2, Counter-timer Hypervisor Control Register”**, in the ‘Field descriptions’ section, the text that reads:

When ELIsInHost(EL2):

is changed to read:

When the Effective value of HCR_EL2.E2H is 1:

Equivalent changes are made in the following sections:

- **D24.2.38 “CPTR_EL2, Architectural Feature Trap Register (EL2)”**.
- **D24.2.191 “TCR2_EL2, Extended Translation Control Register (EL2)”**.
- **D24.2.195 “TCR_EL2, Translation Control Register (EL2)”**.

In sections **D24.2.210 “TTBR0_EL2, Translation Table Base Register 0 (EL2)”** and **D24.2.213 “TTBR1_EL2, Translation Table Base Register 1 (EL2)”**, in the 'Field descriptions' sections, the text that reads:

When FEAT_D128 is implemented, TCR2_EL2.D128 == 1, and ELIsInHost(EL2):

is changed to read:

When FEAT_D128 is implemented, TCR2_EL2.D128 == 1, and the Effective value of HCR_EL2.E2H is 1:

In section **D24.2.175 “SCTLR_EL1, System Control Register (EL1)”**, in the 'MSCEn, bit[33]' field, the text that reads:

When FEAT_MOPS is implemented and !ELIsInHost(EL0):

is changed to read:

When FEAT_MOPS is implemented:

2.15 D23122

In section **D14.3.2 “Common microarchitectural events”**, in the event description of '0x0015, L1D_CACHE_WB, Level 1 data cache write-back' the text that reads:

The counter counts each write-back of data from the Level 1 data or unified cache to outside of the Level 1. For example:

- A write-back of a dirty cache line to a Level 2 cache or memory.
- A write-back of a recently fetched cache line that has not been allocated to the Level 1 data cache.

is changed to read:

The counter counts each write-back of data from the Level 1 data or unified cache to outside of the Level 1. For example:

- A write-back of a cache line to a Level 2 cache or memory.
- A write-back of a recently fetched cache line that has not been allocated to the Level 1 data cache.

In the same event description, the text that reads:

A write-back is attributable to the agent that generated the request that caused the write-back. This might not be the same agent that caused the data being written back to be allocated into the cache.

An Unattributable write-back event occurs when a requestor outside of the PE makes a coherency request that results in write-back. If the cache is shared, then an Unattributable write-back event is not counted. If the cache is not shared, then the event is counted.

is changed to read:

A write-back is attributable to one of the following, and it is **IMPLEMENTATION DEFINED** which:

- The PE or other agent that generated the request that caused the write-back. This might not be the same agent that caused the data to be allocated into the cache.
- If the cache is shared, the PE or other agent that caused the data to be allocated into the cache.
- If the cache is shared, the PE or other agent that last accessed the data when it was in the cache. If the data has not been accessed since being allocated, then the PE or other agent that caused the data to be allocated into the cache.

If the cache is shared, then the write-back event is counted only if it is attributable to the PE counting the event.

If the cache is not shared, then the event is counted by the PE that the cache is attached to, even if the write-back is not attributable to that PE. For example, a requestor outside of the PE made a coherency request that resulted in write-back.

In the same section, in the event description of '0x0046, L1D_CACHE_WB_VICTIM, Level 1 data cache write-back, victim', the text that reads:

The counter counts each write-back counted by L1D_CACHE_WB that occurs because the line is allocated for an access made by the PE.

is changed to read:

The counter counts each write-back counted by L1D_CACHE_WB that occurs because of a capacity eviction due to a line being allocated into the cache.

It is **IMPLEMENTATION DEFINED** whether this includes capacity evictions due to an agent other the PE counting the event. For example, a stashing request from outside of the PE, or an allocation due to another PE that shares the cache.

In all cases, the event is counted only if the eviction is also an L1D_CACHE_WB event. This means that, if the cache is shared, the eviction is counted only if it is attributable to the PE counting the event, as defined by L1D_CACHE_WB.

In the same section, in the event description of '0x0047, L1D_CACHE_WB_CLEAN, Level 1 data cache write-back, cleaning and coherency', the text that reads:

The counter counts each write-back counted by L1D_CACHE_WB that occurs because of a coherency operation made by another PE or, optionally, the execution of a cache maintenance instruction.

Whether write-backs that are caused by the execution of a cache maintenance instruction are counted is **IMPLEMENTATION DEFINED**.

Note:

The transfer of a dirty cache line from the Level 1 data cache of this PE to the data cache of another PE due to a hardware coherency operation is not counted unless the dirty cache line is also written back to a Level 2 cache or memory. If a coherency request from a requestor outside of the PE results in a write-back, it is an Unattributable event.

is changed to read:

The counter counts each write-back counted by L1D_CACHE_WB that occurs because of a coherency operation made by another PE or, optionally, the execution of a cache maintenance instruction.

It is **IMPLEMENTATION DEFINED** whether the transfer of a dirty cache line from the Level 1 data cache of this PE to the data cache of another PE due to a hardware coherency operation is counted when the dirty cache line is not also written back to a Level 2 cache or memory.

In all cases, the event is counted only if the write-back is also an L1D_CACHE_WB event. This means that, if the cache is shared, the write-back is counted only if it is attributable to the PE counting the event, as defined by L1D_CACHE_WB.

Similar updates are made for the following events:

- L2D_CACHE_WB (0x0018)
- L2D_CACHE_VICTIM (0x0056)
- L2D_CACHE_CLEAN (0x0057)
- L3D_CACHE_WB (0x002C)
- L3D_CACHE_VICTIM (0x00A6)
- L3D_CACHE_CLEAN (0x00A7)

2.16 D23136

In section **D6.3.1 “Behavior when address translation is enabled”**, the rule that reads:

R_{SJFRQ}

When the Trace Buffer Unit is enabled, the Trace Buffer Unit might prefetch and cache address translations for the translation regime of the owning Exception level, including when the owning Exception level is out-of-context.

is changed to read:

R_{SJFRQ}

When the Trace Buffer Unit is enabled and collection is not stopped, the Trace Buffer Unit might cause speculative translations for the owning translation regime, including when the owning translation regime is out-of-context.

In section **D17.7 “The Profiling Buffer”**, the following rule is added:

R_{X0001}

When the Profiling Buffer is enabled, profiling is not stopped, and Discard mode is not enabled, the Statistical Profiling Unit might cause speculative translations for the owning translation regime, including when the owning translation regime is out-of-context.

2.17 R23240

In section **D20.5 “Error synchronization event”**, the text that reads:

R_{GRJVN}

An Error synchronization event is generated by any of the following:

- Executing an ESB instruction.
- When FEAT_IESB is implemented, and one of the following is true, taking an exception to an Exception level, ELx, using AArch64:
 - The appropriate SCTL_R_ELx.IESB bit is 0b1.
 - FEAT_DoubleFault is implemented, the Exception level is EL3, and SCR_EL3.NMEA is 0b1.

In Debug state this also applies to executing a DCPSx instruction to ELx.

- When FEAT_IESB is implemented, and one of the following is true, executing an exception return instruction at an Exception level, ELx, using AArch64:
 - The appropriate SCTL_R_ELx.IESB bit is 0b1.
 - FEAT_DoubleFault is implemented, the Exception level is EL3, and SCR_EL3.NMEA is 0b1.

In Debug state this also applies to executing a DRET instruction at ELx.

...

R_{RPBWR}

For each value of ELx in EL1, EL2, EL3, if all of the following are true, then executing a DCPSx instruction generates an Error synchronization event:

- The PE is in Debug state.
- ELx is using AArch64.
- The Effective value of SCTLR_ELx.IESB is 0b1.

...

R_{CVPDN}

For each value of ELx in EL1, EL2, EL3, if all of the following are true, then executing a DRPS instruction at ELx generates an Error synchronization event:

- The PE is in Debug state and the instruction does not generate any exception.
- ELx is using AArch64.
- The Effective value of SCTLR_ELx.IESB is 0b1.

is changed to read:

R_{GRJVN}

An Error synchronization event is generated by any of the following:

- Executing an ESB instruction.
- When FEAT_IESB is implemented, and one of the following is true, taking an exception in Non-debug state to an Exception level, ELx, using AArch64:
 - The appropriate SCTLR_ELx.IESB bit is 0b1.
 - FEAT_DoubleFault is implemented, the Exception level is EL3, and SCR_EL3.NMEA is 0b1.
- When FEAT_IESB is implemented, and one of the following is true, executing an exception return instruction in Non-debug state at an Exception level, ELx, using AArch64:
 - The appropriate SCTLR_ELx.IESB bit is 0b1.
 - FEAT_DoubleFault is implemented, the Exception level is EL3, and SCR_EL3.NMEA is 0b1.

R_{X0001}

SCTLR_ELx.IESB is ignored in Debug state and treated as zero.

I_{X0002}

Because SError exceptions are masked in Debug state, and the effects of an implicit Error synchronization barrier could happen at any time regardless of the value of SCTLR_ELx.IESB, an implementation that honors SCTLR_ELx.IESB in Debug state is compliant with R_{X0001}.

In section **D24.2.175 “SCTLR_EL1, System Control Register (EL1)”**, the text that reads:

When the PE is in Debug state, the effect of this field is **CONSTRAINED UNPREDICTABLE**, and its Effective value might be 0 or 1 regardless of the value of the field.

If the Effective value of the field is 1, then an implicit error synchronization event is added after each DCPSx instruction taken to EL1 and before each DRPS instruction executed at EL1, in addition to the other cases where it is added.

is changed to read:

The value of this field is ignored in Debug state and treated as zero.

Additionally, the same change is made in the following sections:

- **D24.2.176 “SCTLR_EL2, System Control Register (EL2)”**.
- **D24.2.177 “SCTLR_EL3, System Control Register (EL3)”**.

In section **J1.1.1.128 “AArch64.TakeExceptionInDebugState()”**, the following code segment is removed:

```
if IsFeatureImplemented(FEAT_IESB) then
    sync_errors = SCTLR_EL[target_el].IESB == '1';
    if IsFeatureImplemented(FEAT_DoubleFault) then
        sync_errors = sync_errors || (SCR_EL3.<EA,NMEA> == '11' && target_el ==
EL3);
    // The Effective value of SCTLR[].IESB might be zero in Debug state.
    if !ConstrainUnpredictableBool(Unpredictable_IESBinDebug) then
        sync_errors = FALSE;
else
    sync_errors = FALSE;
....
if sync_errors then
    SynchronizeErrors();
```

A similar change is made in section **J1.3.1.53 “DCPSInstruction()”** and section **J1.3.1.54 “DRPSInstruction()”**.

2.18 D23442

In section **D18.2.6 “Events packet”**, under ‘Events packet payload’, in E[25], the text that reads:

E[25], byte 3 bit [1], when SZ == 0b10 or SZ == 0b11

SMCU or external coprocessor operation.

When FEAT_SPE_SME is implemented

E[25]	Description
0b0	Operation did not execute on an SMCU or external coprocessor.
0b1	Operation executed on an SMCU or external coprocessor.

When FEAT_SPEv1p4 is implemented

This bit reads-as-zero.

Otherwise

This bit reads as an **IMPLEMENTATION DEFINED** value.

is changed to read:

E[25], byte 3 bit [1], when SZ == 0b10 or SZ == 0b11

Streaming Mode Compute Unit (SMCU) or other shared resource operation.

When (FEAT_SPE_SME is implemented or FEAT_SPEv1p5 is implemented) and an SMCU or other shared resource is implemented

E[25]	Description
0b0	Operation did not execute on an SMCU or other shared resource.
0b1	Operation executed on an SMCU or other shared resource.

A shared resource means a resource that is shared between PEs for the execution of instructions. It is **IMPLEMENTATION DEFINED** which instructions can be executed on a shared resource.

When FEAT_SPEv1p4 is implemented or FEAT_SPE_SME is implemented

This bit reads-as-zero.

Otherwise

This bit reads as an **IMPLEMENTATION DEFINED** value.

In section **D24.7.11 “PMSEVFR_EL1, Sampling Event Filter Register”**, in E[25], the text that reads:

E[25], bit [25]

When FEAT_SPE_SME is implemented and event 25 is implemented:

Filter on Streaming Mode Compute Unit (SMCU) or external coprocessor operation event.

E[25]	Meaning
0b0	SMCU or external coprocessor operation event is ignored.
0b1	Do not record samples that have the SMCU or external coprocessor operation event == 0.

is changed to read:

E[25], bit [25]

When (FEAT_SPE_SME is implemented or FEAT_SPEv1p5 is implemented) and event 25 is implemented:

Filter on Streaming Mode Compute Unit (SMCU) or other shared resource operation event.

E[25]	Meaning
0b0	SMCU or other shared resource operation event is ignored.
0b1	Do not record samples that have the SMCU or other shared resource operation event == 0.

The equivalent change is made in section **D24.7.17 “PMSNEVFR_EL1, Sampling Inverted Event Filter Register”**.

2.19 R23526

In section **H5.5.2 “CTI authentication”**, the text that reads:

The CTI ignores the state of the **IMPLEMENTATION DEFINED** authentication interface. This means that:

- ...
- The CTI can generate external triggers regardless of the value of `ExternallInvasiveDebugEnabled()`.

Note:

The PE ignores Debug request and Restart request trigger events when `ExternallInvasiveDebugEnabled()` is FALSE. The trace unit ignores Generic trace external input trigger events when `ExternalNoninvasiveDebugEnabled()` is FALSE. The behavior of Generic CTI interrupt requests is part of the **IMPLEMENTATION DEFINED** handling of these interrupts, but it is permissible for an interrupt controller to receive these requests even when `ExternallInvasiveDebugEnabled()` is FALSE.

is changed to read:

The CTI ignores the state of the **IMPLEMENTATION DEFINED** authentication interface. This means that:

- ...
- The CTI can generate external triggers regardless of the values of `ExternallInvasiveDebugEnabled()` and `ExternalNoninvasiveDebugEnabled()`.

Note:

The destination of the trigger is responsible for authenticating the trigger request:

- The PE ignores Debug request trigger events when `ExternallInvasiveDebugEnabled()` is FALSE.
- It is **IMPLEMENTATION DEFINED** whether the PE ignores Restart request trigger events when `ExternallInvasiveDebugEnabled()` is FALSE.
- The trace unit ignores Generic trace external input trigger events when `ExternalNoninvasiveDebugEnabled()` is FALSE.
- The behavior of Generic CTI interrupt requests is part of the **IMPLEMENTATION DEFINED** handling of these interrupts, but it is permissible for an interrupt controller to receive these

requests even when `ExternallInvasiveDebugEnabled()` is `FALSE`, meaning software might need to configure the interrupt controller to ignore these requests.

2.20 C23528

In section **A2.2.10 “The Armv8.9 architecture extension”**, under the subsection ‘FEAT_S1POE, Stage 1 permission overlays’, the text that reads:

FEAT_S1POE allows stage 1 permissions to be progressively restricted by processes running at ELO without requiring TLB maintenance, and reduces the number of calls to privileged software.

is changed to read:

FEAT_S1POE allows stage 1 permissions to be progressively restricted without requiring TLB maintenance. The feature is available at all Exception levels, including ELO, and can reduce the number of calls to privileged software.

2.21 R23639

In section **D6.4.2 “Behavior in External mode”**, the following rule is added:

R_{X0001}

If `TRBPTR_EL1[63:(OAMax+1)]` is nonzero when the Trace Buffer Unit attempts to write to the trace buffer, then one of the following occurs, and it is **CONSTRAINED UNPREDICTABLE** which:

- A stage 1 Address Size fault is generated.
- `TRBPTR_EL1[63:(OAMax+1)]` are ignored and treated as zero.

The value of `OAMax` is as defined by `RMXRFD`.

Additionally, in section **D6.5.4 “Faults”**, the text that reads:

R_{YQMQK}

MMU faults other than GPC faults are not generated in External mode.

is changed to read:

R_{YQMQK}

MMU faults other than GPC faults and, optionally, Address Size faults, are not generated in External mode.

2.22 C23821

In section **D14.3 “Common event numbers”**, the PMU event SVE_PRED_SPEC, previously updated by R23545 to read:

0x8074, SVE_PRED_SPEC, Operation speculatively executed, SVE predicated

The counter counts each speculatively executed predicated SIMD data-processing, load, or store operation. This includes all of the following:

- A data processing or load operation that writes to one or more SVE Z vector destination registers under a Governing predicate using either zeroing or merging predication.
- A predicated store of one or more SVE Z vector registers.

It is **IMPLEMENTATION DEFINED** whether data processing operations due to instructions with a single Governing predicate operand that determines the Active elements that do not write to any SVE Z vector destination registers using either zeroing or merging predication are counted. For example, INCP. Arm recommends these operations are not counted when FEAT_SPE is implemented, for consistency with the Operation Type packet PRED field.

When FEAT_SME is implemented, both operations due to SVE instructions and operations due to SME instructions operating on the SVE Z vectors with at least one Governing predicate operand are counted.

Note:

For outer product instructions which are widening, predication is considered with respect to the input element size.

is changed to read:

0x8074, SVE_PRED_SPEC, Operation speculatively executed, SVE predicated

The counter counts each speculatively executed predicated SIMD data-processing, load, or store operation. This includes all of the following:

- A data processing or load operation that writes to one or more SVE Z vector destination registers under a Governing predicate using either zeroing or merging predication.
- A predicated store of one or more SVE Z vector registers.

It is **IMPLEMENTATION DEFINED** whether data processing operations due to instructions with a single Governing predicate operand that determines the Active elements that do not write to any SVE Z vector destination registers using either zeroing or merging predication are counted. For example, INCP. Arm recommends these operations are not counted when FEAT_SPE is implemented, for consistency with the Operation Type packet PRED field.

Operations where all predicates are FALSE which result in no effective change in architectural state are counted.

When FEAT_SME is implemented, both operations due to SVE instructions and operations due to SME instructions operating on the SVE Z vectors with at least one Governing predicate operand are counted.

Note:

For outer product instructions which are widening, predication is considered with respect to the input element size.

In section **D18.2.7 “Operation Type packet”**, in both cases of the ‘PRED’ field values 0b1, the following text is added:

This includes operations which result in no effective change in architectural state due to all predicates being FALSE.

2.23 R23822

In section **D18.2.7 “Operation Type packet”**, in the description of the ‘PRED’ field, the text that reads:

PRED	Description
0b0	Not predicated.
0b1	Predicated SVE operation. The operation is an SVE operation that writes to a vector destination register under a Governing predicate using either zeroing or merging predication.

is changed to read:

PRED	Description
0b0	Not predicated.
0b1	Predicated SVE data-processing operation.

Predicated SVE data-processing operation means an SVE data-processing operation that writes to one or more SVE Z vector destination registers under a Governing predicate using either zeroing or merging predication. It is **IMPLEMENTATION DEFINED** whether this includes data-processing operations due to instructions with a single Governing predicate operand that determines the Active elements which do not write to any SVE Z vector destination registers using either zeroing or merging predication are counted. For example, INCP.

In section **D18.2.6 “Events packet”**, in the description of the ‘E[18]’ field, the text that reads:

E[18], byte 2 bit [2], when SZ == 0b10 or SZ == 0b11

Empty predicate.

...

is changed to read:

E[18], byte 2 bit [2], when SZ == 0b10 or SZ == 0b11

Empty predicate.

...

For a sampled SVE operation, it is **IMPLEMENTATION DEFINED** whether this field is valid or reads as zero when the PRED field in the Operation Type field is 0.

The equivalent change is made to the E[17] “Partial or empty predicate” field.

2.24 C23965

In section **D24.2.90 “ID_AA64MMFR2_EL1, AArch64 Memory Model Feature Register 2”**, the field description for EVT, bits [59:56], the text that reads:

EVT, bits [59:56]

Enhanced Virtualization Traps. If EL2 is implemented, indicates support for the HCR_EL2.{TTLBOS, TTLBIS, TOCU, TICAB, TID4} traps.

The value of this field is an **IMPLEMENTATION DEFINED** choice of:

EVT	Meaning
0b0000	HCR_EL2.{TTLBOS, TTLBIS, TOCU, TICAB, TID4} traps are not supported.
0b0001	HCR_EL2.{TOCU, TICAB, TID4} traps are supported. HCR_EL2.{TTLBOS, TTLBIS} traps are not supported.
0b0010	HCR_EL2.{TTLBOS, TTLBIS, TOCU, TICAB, TID4} traps are supported.

All other values are reserved.

FEAT_EVT implements the functionality identified by the values 0b0001 and 0b0010.

If EL2 is not implemented, the only permitted value is 0b0000.

From Armv8.5, if EL2 is implemented, the value 0b0001 is not permitted.

Access to this field is RO.

is changed to read:

EVT, bits [59:56]

Enhanced Virtualization Traps. If EL2 is implemented, indicates support for the HCR_EL2.{TTLBOS, TTLBIS, TOCU, TICAB, TID4} traps.

The value of this field is an **IMPLEMENTATION DEFINED** choice of:

EVT	Meaning
0b0000	HCR_EL2.{TTLBOS, TTLBIS, TOCU, TICAB, TID4} traps are not supported.
0b0001	HCR_EL2.{TOCU, TICAB, TID4} traps are supported.
0b0010	As 0b0001, and HCR_EL2.{TTLBOS, TTLBIS} traps are supported.

All other values are reserved.

FEAT_EVT implements the functionality identified by the value 0b0001.

FEAT_EVT2 implements the functionality identified by the value 0b0010.

If EL2 is not implemented, the only permitted value is 0b0000.

From Armv8.5, if EL2 is implemented, the value 0b0001 is not permitted.

Access to this field is RO.

Equivalent changes are made in the following sections:

- **D24.2.112 “ID_MMFR4_EL1, AArch32 Memory Model Feature Register 4”.**
- **G8.2.97 “ID_MMFR4, Memory Model Feature Register 4”.**

In section **A2.2.6 “The Armv8.5 architecture extension”**, FEAT_EVT2 is added:

FEAT_EVT2, Enhanced Virtualization Traps 2

FEAT_EVT2 extends FEAT_EVT to introduce additional traps for further EL1 and ELO system control operations in:

- HCR_EL2.
- HCR2.

FEAT_EVT2 is OPTIONAL from Armv8.2.

In an Armv8.5 implementation, if FEAT_EL2 is implemented, FEAT_EVT2 is implemented.

If FEAT_EVT2 is implemented, then FEAT_EVT is implemented.

The following fields identify the presence of FEAT_EVT2:

- ID_AA64MMFR2_EL1.EVT.
- ID_MMFR4_EL1.EVT.
- ID_MMFR4.EVT.

In section **D24.2.62 “HCR_EL2, Hypervisor Configuration Register”**, in the field conditions of fields ‘TTLBOS, bit [55]’ and ‘TTLBIS, bit [54]’, the text that reads:

When FEAT_EVT is implemented:

is changed to read:

When FEAT_EVT2 is implemented:

In section **D24.2.45 “ESR_EL1, Exception Syndrome Register (EL1)”**, under the subheading ‘Additional Information for ISS encoding for an exception from MSR, MRS, or System instruction execution in AArch64 state’, the text that reads:

If FEAT_EVT is implemented, the following registers control traps for EL1 and EL0 Cache controls that use this EC value:

- HCR_EL2.{TTLBOS, TTLBIS, TICAB, TOCU, TID4}.
- HCR2.{TTLBIS, TICAB, TOCU, TID4}.

is changed to read:

If FEAT_EVT is implemented, the following registers control traps for EL1 and EL0 registers that use this EC value:

- HCR_EL2.{TICAB, TOCU, TID4}.
- HCR2.{TICAB, TOCU, TID4}.

If FEAT_EVT2 is implemented, the following registers control traps for EL1 and EL0 instructions that use this EC value:

- HCR_EL2.{TTLBOS, TTLBIS}.

If FEAT_EVT2 is implemented, HCR2.TTLBIS controls traps for EL1 and EL0 instructions using AArch32 state that use this EC value.

Equivalent changes are made in the following sections:

- **D24.2.46 “ESR_EL2, Exception Syndrome Register (EL2)”**.
- **D24.2.47 “ESR_EL3, Exception Syndrome Register (EL3)”**.

2.25 C24066

In section **D14.3.1 “Common architectural events”**, in event “0x8000, SIMD_INST_RETIRED, Instruction architecturally executed, SIMD”, the text that reads:

0x8000, SIMD_INST_RETIRED, Instruction architecturally executed, SIMD

The counter counts each architecturally executed SIMD instruction.

It is **IMPLEMENTATION DEFINED** which architecturally executed SIMD instructions are counted in AArch32 state.

When Armv9.5 is not implemented, it is **IMPLEMENTATION DEFINED** whether scalar loads and stores to SIMD&FP registers other than those listed above are counted.

When Armv9.5 is implemented, other scalar loads and stores to SIMD&FP registers are not counted.

is changed to read:

0x8000, SIMD_INST_RETIRED, Instruction architecturally executed, SIMD

The counter counts each architecturally executed instruction counted by INST_RETIRED that is a SIMD instruction.

That is, the counter counts:

- Advanced SIMD data-processing instructions. Advanced SIMD scalar data-processing instructions are not counted.
- SVE and SME SIMD data-processing instructions. Non-SIMD SVE and SME data-processing instructions are not counted.
- Advanced SIMD structure load/store of one or more SIMD&FP registers instructions.
- Advanced SIMD load and replicate to one or more SIMD&FP registers instructions.
- Scalar load/store of a SIMD&FP Q register or pair of Q registers instructions.
- SVE and SME load/store instructions.

It is **IMPLEMENTATION DEFINED** which architecturally executed SIMD instructions are counted in AArch32 state.

When Armv9.5 is not implemented, it is **IMPLEMENTATION DEFINED** whether scalar loads and stores to SIMD&FP registers other than those listed above are counted.

When Armv9.5 is implemented, other scalar loads and stores to SIMD&FP registers are not counted.

2.26 C24074

In section **B2.3 “Ordering requirements defined by the formal concurrency model”**, the following text is added:

For the purposes of the memory model, all Effects generated due to execution of an instruction are considered to be generated by the PE on which the instruction is executed. The list of all Effects of an instruction in the scope of the Arm Memory Model is further presented in Effects.

Additionally, in the **“Glossary”**, the definition of ‘Observer’ that reads:

Observer

A PE or mechanism in the system, such as a peripheral device, that can generate reads from or writes to memory.

is changed to read:

Observer

An observer is one of:

- The mechanism inside a PE that generates Implicit Translation Table Descriptor (TTD) Memory Effects
- The mechanism inside a PE that generates Implicit Instruction Memory Read Effects
- The mechanism inside a PE that generates Explicit Memory Effects, Tag Memory Effects, Barrier Effects, Instruction Fetch Barrier Effects, TLBI Effects, DC Effects, IC Effects, Fault Effects, GCS Effects.
- The mechanism inside the PE that generates Memory Effects for SPE.
- The mechanism inside the PE that generates Memory Effects for TRBE.
- The mechanism inside a PE that generates Memory Effects for HDBSS.
- The mechanism inside a PE that generates Memory Effects for HACDBS.
- Any mechanism outside the PE that can generate Memory Effects, for example, a peripheral device, an SMMU, or a GIC.

Additionally, in section **B2.4.1 “Completion and endpoint ordering”**, the text that reads:

For all memory, the completion rules are defined as:

- A Memory Read effect R_1 to a Location is complete for a shareability domain when all of the following are true:
 - Any write to the same Location by an observer within the shareability domain will be Coherence-after R_1 .
 - Any translation table walks associated with R_1 are complete for that shareability domain.
- A Memory Write effect W_1 to a Location is complete for a shareability domain when all of the following are true:
 - Any write to the same Location by an observer within the shareability domain will be Coherence-after W_1 .
 - Any read to the same Location by an observer within the shareability domain will either Reads-from W_1 or Reads-from a Memory Write effect that is Coherence-after W_1 .
 - Any translation table walks associated with the write are complete for that shareability domain.
- A translation table walk is complete for a shareability domain when the memory accesses, including the updates to translation table entries, associated with the translation table walk are complete for that shareability domain, and the TLB is updated.
- A cache maintenance instruction is complete for a shareability domain when the memory effects of the instruction are complete for that shareability domain, and any translation table walks that arise from the instruction are complete for that shareability domain.

is changed to read:

For all memory, the completion rules are defined as:

- A Memory Read effect E_1 to a Location is complete for a shareability domain when all of the following are true:
 - Any Memory Write Effect to the same Location by an observer within the shareability domain will be Coherence-after E_1 .
 - All Implicit TTD Memory Read Effects associated with E_1 are complete for that shareability domain.
- A Memory Write effect E_1 to a Location is complete for a shareability domain when all of the following are true:
 - Any Memory Write Effect to the same Location by an observer within the shareability domain will be Coherence-after E_1 .
 - Any Memory Read Effect to the same Location by an observer within the shareability domain will either Reads-from E_1 or Reads-from a Memory Write effect that is Coherence-after E_1 .
 - All Implicit TTD Memory Read Effects associated with E_1 are complete for that shareability domain.
- An Implicit TTD Memory Read Effect is complete for a shareability domain when the memory accesses associated with the Implicit TTD Memory Read Effect, including Hardware Update Effects, are complete for that shareability domain, and the TLB is updated.
- A cache maintenance instruction is complete for a shareability domain when the memory effects of the instruction are complete for that shareability domain, and any Implicit TTD Memory Read Effects that arise from the instruction are complete for that shareability domain.

2.27 C24078

In section **D24.2.96 “ID_AA64SMFR0_EL1, SME Feature ID Register 0”**, the text that reads:

SMEver, bits [59:56]

Indicates support for SME instructions when FEAT_SME is implemented.

The value of this field is an **IMPLEMENTATION DEFINED** choice of:

SMEver	Meaning
0b0000	The mandatory SME instructions are implemented.
0b0001	As 0b0000, and adds the mandatory SME2 instructions.
0b0010	As 0b0001, and adds the mandatory SME2.1 instructions.

All other values are reserved.

FEAT_SME implements the functionality identified by the value 0b0000.

FEAT_SME2 implements the functionality identified by the value 0b0001.

FEAT_SME2p1 implements the functionality identified by the value 0b0010.

From Armv9.4, the value 0b0001 is not permitted.

is changed to to read:

SMEver, bits [59:56]

When FEAT_SME is implemented

Indicates support for SME instructions.

The value of this field is an **IMPLEMENTATION DEFINED** choice of:

SMEver	Meaning
0b0000	The mandatory SME instructions are implemented.
0b0001	As 0b0000, and adds the mandatory SME2 instructions.
0b0010	As 0b0001, and adds the mandatory SME2.1 instructions.

All other values are reserved.

FEAT_SME implements the functionality identified by the value 0b0000.

FEAT_SME2 implements the functionality identified by the value 0b0001.

FEAT_SME2p1 implements the functionality identified by the value 0b0010.

From Armv9.4, the value 0b0001 is not permitted.

Otherwise

Reserved, **RES0**.

In section **D24.2.97 “ID_AA64ZFR0_EL1, SVE Feature ID Register 0”**, the text that reads:

SVEver, bits [3:0]

Indicates support for SVE instructions when FEAT_SME or FEAT_SVE is implemented.

The value of this field is an **IMPLEMENTATION DEFINED** choice of:

SVEver	Meaning
0b0000	The SVE instructions are implemented.
0b0001	As 0b0000, and adds the mandatory SVE2 instructions.
0b0010	As 0b0001, and adds the mandatory SVE2.1 instructions.

All other values are reserved.

FEAT_SVE2 implements the functionality identified by 0b0001 when the PE is not in Streaming SVE mode.

FEAT_SME implements the functionality identified by 0b0001 when the PE is in Streaming SVE mode.

FEAT_SME2p1 implements the functionality identified by 0b0010 when the PE is in Streaming SVE mode.

FEAT_SVE2p1 implements the functionality identified by 0b0010 when the PE is not in Streaming SVE mode.

From Armv9, if this register is present, the value 0b0000 is not permitted.

From Armv9.4, the value 0b0001 is not permitted. Access to this field is RO.

is changed to read:

SVEver, bits [3:0]

When FEAT_SVE is implemented or FEAT_SME is implemented

Indicates support for SVE instructions.

The value of this field is an **IMPLEMENTATION DEFINED** choice of:

SVEver	Meaning
0b0000	The SVE instructions are implemented.
0b0001	As 0b0000, and adds the mandatory SVE2 instructions.
0b0010	As 0b0001, and adds the mandatory SVE2.1 instructions.

All other values are reserved.

FEAT_SVE2 implements the functionality identified by 0b0001 when the PE is not in Streaming SVE mode.

FEAT_SME implements the functionality identified by 0b0001 when the PE is in Streaming SVE mode.

FEAT_SME2p1 implements the functionality identified by 0b0010 when the PE is in Streaming SVE mode.

FEAT_SVE2p1 implements the functionality identified by 0b0010 when the PE is not in Streaming SVE mode.

From Armv9, the value 0b0000 is not permitted.

From Armv9.4, the value 0b0001 is not permitted.

Access to this field is RO.

Otherwise

Reserved, **RES0**.

2.28 D24095

In section **J1.1.4 “aarch64/translation”**, in the function `AArch64.S1IndirectBasePermissions()`, the code that reads:

```

if HasUnprivileged(regime) then
    case permissions.upi of
        ...

        ...

    // If the decoded permissions has either px or pgcs along with either uw or
ugcs,
    // then all effective Stage 1 Base Permissions are set to 0
    if ((px == '1' || pgcs == '1') && (uw == '1' || ugcs == '1')) then
        (pr,pw,px,pgcs) = ('0','0','0','0');
        (ur,uw,ux,ugcs) = ('0','0','0','0');

    if (IsFeatureImplemented(FEAT_PAN) && accdesc.pan && !(regime == Regime_EL10
&&
        walkparams.nvl == '1')) then
        if PSTATE.PAN == '1' && (permissions.upi != '0000') then
            (pr,pw) = ('0','0');
```

is changed to read:

```

if HasUnprivileged(regime) then
    case permissions.upi of
        ...

        ...

    // If the decoded permissions has either px or pgcs along with either uw or
ugcs,
    // then all effective Stage 1 Base Permissions are set to 0
    if ((px == '1' || pgcs == '1') && (uw == '1' || ugcs == '1')) then
        (pr,pw,px,pgcs) = ('0','0','0','0');
        (ur,uw,ux,ugcs) = ('0','0','0','0');
    elsif (IsFeatureImplemented(FEAT_PAN) && accdesc.pan && !(regime ==
Regime_EL10 &&
        walkparams.nvl == '1')) then
        if (boolean IMPLEMENTATION_DEFINED "Realm EL2&0 regime affects EPAN") &&
            accdesc.ss == SS_Realm && regime == Regime_EL20 &&
            walkstate.baseaddress.paspace != PAS_Realm) then
            ux = '0';

        if (boolean IMPLEMENTATION_DEFINED "SCR_EL3.SIF affects EPAN" &&
            accdesc.ss == SS_Secure && walkstate.baseaddress.paspace ==
PAS_NonSecure &&
            walkparams.sif == '1') then
            ux = '0';

        if PSTATE.PAN == '1' && (!IsZero(ur:ur:ux:ugcs) || (!
IsZero(permissions.upi) &&
            boolean IMPLEMENTATION_DEFINED "PAN applies when S1UnprivBasePerm
is reserved")) then
            (pr,pw) = ('0','0');
```

2.29 C24109

In section **B2.3.7 “Ordering relations”**, in the definition of DSB-ordered-before, the text that reads:

An Effect E_1 is DSB-ordered-before an Effect E_2 if one of the following applies:

- ...
- All of the following apply:
 - FEAT_ETS2 is implemented.
 - One of the following applies:
 - E_1 is a Memory Effect.
 - E_1 is a DC CVAU Effect.
 - E_1 is an IC Effect.
 - E_1 appears in program order before E_3 .
 - E_3 is a DSB FULL Effect.
 - E_3 appears in program order before E_2 .
 - E_2 is an Implicit TTD Memory Effect.
- ...
- All of the following apply:
 - FEAT_ETS2 is implemented.
 - E_1 is an Explicit Memory Read Effect.
 - It is not the case that E_1 is generated by an instruction whose destination register is WZR or XZR.
 - E_1 appears in program order before E_3 .
 - E_3 is a DSB LD Effect.
 - E_3 appears in program order before E_2 .
 - E_2 is an Implicit TTD Memory Effect.
- ...
- All of the following apply:
 - FEAT_ETS2 is implemented.
 - E_1 is an Explicit Memory Write Effect.
 - E_1 appears in program order before E_3 .
 - E_3 is a DSB ST Effect.
 - E_3 appears in program order before E_2 .
 - E_2 is an Implicit TTD Memory Effect.

is changed to read:

An Effect E_1 is DSB-ordered-before an Effect E_2 if one of the following applies:

- ...
- All of the following apply:
 - FEAT_ETS2 is implemented.
 - One of the following applies:
 - E_1 is a Memory Effect.
 - E_1 is a DC CVAU Effect.
 - E_1 is an IC Effect.
 - E_1 appears in program order before E_3 .
 - E_3 is a DSB FULL Effect.
 - E_3 appears in program order before E_2 .
 - E_2 is an Implicit TTD Memory Effect.
- ...
- All of the following apply:
 - FEAT_ETS2 is implemented.
 - E_1 is an Explicit Memory Read Effect.
 - It is not the case that E_1 is generated by an instruction whose destination register is WZR or XZR.
 - E_1 appears in program order before E_3 .
 - E_3 is a DSB LD Effect.
 - E_3 appears in program order before E_2 .
 - E_2 is an Implicit TTD Memory Effect.
- ...
- All of the following apply:
 - FEAT_ETS2 is implemented.
 - E_1 is an Explicit Memory Write Effect.
 - E_1 appears in program order before E_3 .
 - E_3 is a DSB ST Effect.
 - E_3 appears in program order before E_2 .
 - E_2 is an Implicit TTD Memory Effect.

Note:

Regardless of whether FEAT_ETS2 is implemented, the DSB-ordered-before relation does not apply when E_2 is an Implicit TTD Memory Read Effect generated for an instruction fetch.

In the same section, the definition of TTD-read-ordered-before is extended with a similar note. The text that reads:

An Effect E_1 is TTD-read-ordered-before an Effect E_2 if one of the following applies:

- ...
- All of the following apply:
 - FEAT_ETS2 is implemented.
 - E_1 is TLBI-before E_3 .
 - E_3 is a TLBI Effect.
 - E_3 appears in program order before E_4 .
 - E_4 is a DSB FULL Effect.
 - E_4 appears in program order before E_2 .
 - E_2 is an Implicit TTD Memory Effect.

is changed to read:

An Effect E_1 is TTD-read-ordered-before an Effect E_2 if one of the following applies:

- ...
- All of the following apply:
 - FEAT_ETS2 is implemented.
 - E_1 is TLBI-before E_3 .
 - E_3 is a TLBI Effect.
 - E_3 appears in program order before E_4 .
 - E_4 is a DSB FULL Effect.
 - E_4 appears in program order before E_2 .
 - E_2 is an Implicit TTD Memory Effect.

Note:

Regardless of whether FEAT_ETS2 is implemented, the TTD-read-ordered-before relation does not apply when E_2 is an Implicit TTD Memory Read Effect generated for an instruction fetch.

2.30 D24150

In section **K7.3 “Recommended authentication interface”**, the text that reads:

If EL3 is not implemented and the PE is in Non-secure state, SPIDEN and SPNIDEN are not implemented, and the PE behaves as if these signals were tied LOW.

If EL3 is not implemented and the PE is in Secure state, SPIDEN is usually connected to DBGEN and SPNIDEN is connected to NIDEN, but this is not required. The recommended interface is defined as if all four signals are implemented.

is changed to read:

If FEAT_Secure is not implemented, then all of the following apply:

- SPIDEN and SPNIDEN are not implemented.
- If FEAT_RME is implemented, then the PE behaves as if SPIDEN has the same value as RTPIDEN.
- If FEAT_RME is not implemented, then the PE behaves as if SPIDEN is tied LOW.
- The PE behaves as if SPNIDEN is tied LOW.

If FEAT_RME is not implemented, then RLPIDEN and RTPIDEN are not implemented, and the PE behaves as if these signals were tied LOW.

If EL3 is not implemented and FEAT_Secure is implemented, then it is often the case that SPIDEN is connected to DBGEN and SPNIDEN is connected to NIDEN, but this is not required. The recommended interface is defined as if all four signals are implemented.

In section **J1.3.1.24 “ExternalSecureInvasiveDebugEnabled”**, the code that reads:

```
boolean ExternalSecureInvasiveDebugEnabled()
    if !HaveSecureState() then return FALSE;
    return ExternalInvasiveDebugEnabled() && SPIDEN == HIGH;
```

is changed to read:

```
boolean ExternalSecureInvasiveDebugEnabled()
    if !HaveSecureState() then
        return (!IsFeatureImplemented(FEAT_RME) || RTPIDEN == HIGH);
    return ExternalInvasiveDebugEnabled() && SPIDEN == HIGH;
```

In section **D3.1.2 “Register controls to enable self-hosted trace”**, the text that reads:

If FEAT_TRF is implemented, self-hosted trace is enabled if any of the following are true:

- EDSCR.TFO == 0.
- EDSCR.TFO == 1, EL3 is implemented, MDCR_EL3.STE == 1, and ExternalSecureNoninvasiveDebugEnabled() == FALSE.
- EDSCR.TFO == 1, EL3 is not implemented, the PE executes in Secure state, and ExternalSecureNoninvasiveDebugEnabled() == FALSE.
- EDSCR.TFO == 1, FEAT_RME is implemented, MDCR_EL3.RLTE == 1, and ExternalRealmNoninvasiveDebugEnabled() == FALSE.

The pseudocode function SelfHostedTraceEnabled() shows these rules.

is changed to read:

If FEAT_TRF is implemented, self-hosted trace is enabled if any of the following are true:

- EDSCR.TFO is 0.

- EDSCR.TFO is 1, EL3 is implemented, Secure state is implemented, MDCR_EL3.STE is 1, and ExternalSecureNoninvasiveDebugEnabled() == FALSE.
- EDSCR.TFO is 1, EL3 is not implemented, Secure state is implemented, and ExternalSecureNoninvasiveDebugEnabled() == FALSE.
- EDSCR.TFO is 1, FEAT_RME is implemented, MDCR_EL3.RLTE is 1, and ExternalRealmNoninvasiveDebugEnabled() == FALSE.

The pseudocode function SelfHostedTraceEnabled() shows these rules.

2.31 D24156

In section **H9.2.7 “DBGDTRTX_EL0, Debug Data Transfer Register, Transmit”** the text that reads:

When DoubleLockStatus(), or !IsCorePowered(), or OSLockStatus(), or !AllowExternalDebugAccess(), accesses to this register return an ERROR.

is changed to read:

When DoubleLockStatus(), or !IsCorePowered(), or OSLockStatus(), accesses to this register return an ERROR.

In section **J1.3.1 shared/debug**, the pseudocode that reads:

```
// AllowExternalDebugAccess()
// =====
// Returns TRUE if an external debug interface access to the External debug
// registers
// is allowed for the given Security state, FALSE otherwise.
```

is changed to read:

```
// AllowExternalDebugAccess()
// =====
// Returns TRUE if an external debug interface access to the External
// DBGBVR<n>_EL1,
// DBGBCR<n>_EL1, DBGWVR<n>_EL1, DBGWCR<n>_EL1 registers, and, from Armv8.2, the
// OSLAR_EL1 register is allowed for the access. Returns FALSE otherwise.
```

2.32 D24241

In section **D24.10.2 “CNTHCTL_EL2, Counter-timer Hypervisor Control Register”**, the text in the Configuration section, that reads:

This register has no effect if EL2 is not enabled in the current Security state.

is changed to read:

Fields that control the generation of events from the event stream have an effect regardless of the current Exception level and whether EL2 is enabled in the current Security state.

All other fields have no effect if EL2 is not enabled in the current Security state.

2.33 D24315

In section **D14.3.1 “Common architectural events”**, the L1D_CACHE_LMISS_RD event description that reads:

0x0039, L1D_CACHE_LMISS_RD, Level 1 data cache long-latency read miss

The counter counts each Memory-read operation to the Level 1 data or unified cache counted by L1D_CACHE that incurs additional latency because it returns data from outside of the Level 1 data or unified cache of this PE.

is changed to read:

0x0039, L1D_CACHE_LMISS_RD, Level 1 data cache long-latency read miss

The counter counts each access to the Level 1 data or unified cache counted by L1D_CACHE_RD that incurs additional latency because it returns data from outside of the Level 1 data or unified cache of this PE.

Equivalent changes are made to the L2D_CACHE_LMISS_RD and L3D_CACHE_LMISS_RD events.

2.34 R24327

In section **D24.2.46 “ESR_EL2, Exception Syndrome Register (EL2)”**, in the description of the field ISV under the subsection “ISS encoding for an exception from a Data Abort”, the following statement is added:

For cases where ISV would otherwise be set to 1, it is permitted but not required for ISV to be reported as 0 when the value of FAR_EL2 does not correspond to the lowest address accessed by the instruction. In these cases, Arm recommends reporting ISV as 0.

The equivalent changes are made in the following sections:

- **D24.2.45 “ESR_EL1, Exception Syndrome Register (EL1)”**.
- **D24.2.47 “ESR_EL3, Exception Syndrome Register (EL3)”**.

2.35 D24361

In section **J1.1.4 aarch64/translation**, in function `AArch64.S1Translate()`, the code that reads:

```
// Either the access flag was clear or AP[2]/nDirty is set
if new_desc != descriptor then
    AddressDescriptor descaddr;
    descaccess = CreateAccDescTTEUpdate(accdesc);
```

is changed to read:

```
// Either the access flag was clear or AP[2]/nDirty was set
if new_desc != descriptor then
    if (!IsWBShareable(descipaddr.memattr) &&
        ConstrainUnpredictableBool(Unpredictable_Unsupported_Atomic_HW_Update)) then
        fault.statuscode = Fault_HWUpdateAccessFlag;
        return (fault, AddressDescriptor UNKNOWN);
    AddressDescriptor descaddr;
    descaccess = CreateAccDescTTEUpdate(accdesc);
```

In the same section, in function `AArch64.S2Translate()`, the code that reads:

```
// Either the access flag was clear or S2AP[1]/Dirty is clear
if new_desc != descriptor then
    if walkparams.hdbss == '1' && descriptor<7> == '0' && new_desc<7> == '1' then
        fault = AppendToHDBSS(fault, ipa.paddress, accdesc, walkparams,
            walkstate.level);
```

is changed to read:

```
// Either the access flag was clear or S2AP[1]/Dirty was clear
if new_desc != descriptor then
    if (!IsWBShareable(descipaddr.memattr) &&
        ConstrainUnpredictableBool(Unpredictable_Unsupported_Atomic_HW_Update)) then
        fault.statuscode = Fault_HWUpdateAccessFlag;
        return (fault, AddressDescriptor UNKNOWN);
    if walkparams.hdbss == '1' && descriptor<7> == '0' && new_desc<7> == '1' then
        fault = AppendToHDBSS(fault, ipa.paddress, accdesc, walkparams,
            walkstate.level);
```

And the code that reads:

```
if (accdesc.atomicop && !IsWBShareable(s2_memattr) &&
    ConstrainUnpredictableBool(Unpredictable_Atomic_MMU_IMPDEF_FAULT)) then
    fault.statuscode = Fault_Exclusive;
    return (fault, pa);
```

is changed to read:

```
if (accdesc.acctype == AccessType_TTW && accdesc.atomicop && !
    IsWBShareable(pa.memattr) &&
    ConstrainUnpredictableBool(Unpredictable_Unsupported_Atomic_HW_Update)) then
    fault.statuscode = Fault_HWUpdateAccessFlag;
    return (fault, pa);
```

```
if (accdesc.acctype != AccessType_TTW && accdesc.atomicop && !
    IsWBShareable(pa.memattrs) &&
    ConstrainUnpredictableBool(Unpredictable_Atomic_MMU_IMPDEF_FAULT)) then
    fault.statuscode = Fault_Exclusive;
    return (fault, pa);
```

In function `AArch64.S1Walk()`, the code that reads:

```
if new_descriptor != descriptor then
    AddressDescriptor descaddr;
    constant AccessDescriptor descaccess = CreateAccDescTTEUpdate(accdesc);
```

is changed to read:

```
if new_descriptor != descriptor then
    if (!IsWBShareable(walkaddress.memattrs) &&
        ConstrainUnpredictableBool(Unpredictable_Unsupported_Atomic_HW_Update)) then
        fault.statuscode = Fault_HWUpdateAccessFlag;
        return (fault, AddressDescriptor UNKNOWN, TTWState UNKNOWN, bits(N) UNKNOWN);
    AddressDescriptor descaddr;
    constant AccessDescriptor descaccess = CreateAccDescTTEUpdate(accdesc);
```

In function `AArch64.S2Walk()`, the code that reads:

```
if new_descriptor != descriptor then
    constant AccessDescriptor descaccess = CreateAccDescTTEUpdate(accdesc);
    (fault, descriptor) = AArch64.MemSwapTableDesc(fault, descriptor, new_descriptor,
                                                    walkparams.ee, descaccess,
                                                    walkaddress, N);
```

is changed to read:

```
if new_descriptor != descriptor then
    if (!IsWBShareable(walkaddress.memattrs) &&
        ConstrainUnpredictableBool(Unpredictable_Unsupported_Atomic_HW_Update)) then
        fault.statuscode = Fault_HWUpdateAccessFlag;
        return (fault, AddressDescriptor UNKNOWN, TTWState UNKNOWN, bits(N) UNKNOWN);
    constant AccessDescriptor descaccess = CreateAccDescTTEUpdate(accdesc);
    (fault, descriptor) = AArch64.MemSwapTableDesc(fault, descriptor, new_descriptor,
                                                    walkparams.ee, descaccess,
                                                    walkaddress, N);
```

2.36 D24371

This issue has been fixed in the most recent release of the *Arm® Architecture Reference Manual for A-profile architecture* and was incorrectly mentioned in the previous release of this document. This issue will therefore be removed from future releases of the *Arm® Architecture Reference Manual for A-profile architecture: Known issues*.

2.37 R24482

In section **C6.2.119 “CPYP, CPYM, CPYE”**, the text that reads:

For CPYE, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn.
 - Xd holds the lowest address to be copied to - Xn.
 - On completion of the instruction, Xn holds 0.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn.
 - Xd holds the highest address to be copied to + 1 - Xn.
 - On completion of the instruction, Xn holds 0.

For CPYE, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction (PSTATE.N == 1), then: - Xs holds the highest address to be copied from. - Xd holds the highest address to be copied to. - On completion of the instruction: - Xn holds 0. - Xs holds the highest address that has not been copied from + 1. - Xd holds the highest address that has not been copied to + 1.

is changed to read:

For CPYE, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn.
 - Xd holds the lowest address to be copied to - Xn.

- If the copy is in the backward direction (X_n holds a positive number), then:
 - X_n holds the number of bytes remaining to be copied.
 - X_s holds the highest address to be copied from $+1 - X_n$.
 - X_d holds the highest address to be copied to $+1 - X_n$.
- On completion of the instruction:
 - X_n holds 0.
 - X_s holds an **IMPLEMENTATION SPECIFIC** value in the range with bounds:
 - the lowest address that has been copied from.
 - the highest address that has been copied from $+1$.
 - X_d holds an **IMPLEMENTATION SPECIFIC** value in the range with bounds:
 - the lowest address that has been copied to.
 - the highest address that has been copied to $+1$.

For CPYE, option B, when $PSTATE.C = 1$:

- X_n holds the number of bytes remaining to be copied.
- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - X_s holds the lowest address to be copied from.
 - X_d holds the lowest address to be copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - X_s holds the highest address to be copied from.
 - X_d holds the highest address to be copied to.
- On completion of the instruction:
 - X_n holds 0.
 - X_s holds an **IMPLEMENTATION SPECIFIC** value in the range with bounds:
 - the lowest address that has been copied from.
 - the highest address that has been copied from $+1$.
 - X_d holds an **IMPLEMENTATION SPECIFIC** value in the range with bounds:
 - the lowest address that has been copied to.
 - the highest address that has been copied to $+1$.

Equivalent changes are made to the following sections:

- **C6.2.120 “CPYPN, CPYMN, CPYEN”.**
- **C6.2.121 “CPYPRN, CPYMRN, CPYERN”.**
- **C6.2.122 “CPYPRT, CPYMRT, CPYERT”.**
- **C6.2.123 “CPYPRTN, CPYMRTN, CPYERTN”.**
- **C6.2.124 “CPYPRTRN, CPYMRTRN, CPYERTRN”.**

- C6.2.125 “CPYPRTWN, CPYMRTWN, CPYERTWN”.
- C6.2.126 “CPYPT, CPYMT, CPYET”.
- C6.2.127 “CPYPTN, CPYMTN, CPYETN”.
- C6.2.128 “CPYPTRN, CPYMTRN, CPYETRN”.
- C6.2.129 “CPYPTWN, CPYMTWN, CPYETWN”.
- C6.2.130 “CPYPWN, CPYMWN, CPYEWN”.
- C6.2.131 “CPYPWT, CPYMW, CPYEWT”.
- C6.2.132 “CPYPWTN, CPYMW, CPYEWTN”.
- C6.2.133 “CPYPWTRN, CPYMWTRN, CPYEWTRN”.
- C6.2.134 “CPYPWTWN, CPYMW, CPYEWTWN”.

2.38 C24523

In section **D10.3 “Memory region tagging types”**, the text that reads:

R_{JJRP}

The memory region tagging type for a memory region is:

- Tagged if all of the following are true:
- ...
- Otherwise it is Canonically Tagged if all of the following are true:
 - FEAT_MTE_CANONICAL_TAGS is implemented.
 - Canonical Tagging is enabled for the memory region.
 - The stage 1 attributes do not define the Tagged attribute.
- Otherwise it is Untagged

R_{XNXP}

If FEAT_MTE_CANONICAL_TAGS is implemented, Canonical Tagging is configured by the following controls:

- If the stage 1 translation supports two VA ranges, SCTLR_ELx.{ATA, ATA0} and TCR_ELx.{MTXn, TBI}.
- If the stage 1 translation supports a single VA range, SCTLR_ELx.ATA and TCR_ELx.{MTX, TBI}.

is changed to read:

R_{JJRP}

The memory region tagging type for a memory access is:

- Tagged if all of the following are true:
- ...
- Otherwise it is Canonically Tagged if all of the following are true:
 - FEAT_MTE_CANONICAL_TAGS is implemented.
 - Allocation Tag Access is enabled for the memory access.
 - Canonical Tagging is enabled.
 - The stage 1 attributes do not define the Tagged attribute.
- Otherwise it is Untagged

R_{XNJP}

If FEAT_MTE_CANONICAL_TAGS is implemented, Canonical Tagging is configured by the following controls:

- If the stage 1 translation supports two VA ranges, TCR_ELx.MTXn.
- If the stage 1 translation supports a single VA range, TCR_ELx.MTX.

Additionally, the text in **D24.2.194 “TCR_EL1, Translation Control Register (EL1)”**, field MTX1, that reads:

MTX1	Meaning
0b0	This control has no effect on the PE.
0b1	Bits[59:56] of a 64-bit VA hold a Logical Address Tag, and all of the following apply: • Bits[59:56] are treated as 0b1111 when checking if the address is out of range. • If FEAT_PAuth is implemented, bits[59:56] are not part of the PAC field. • A Canonical Tag Check operation is performed on Tag Checked memory accesses to a Canonically Tagged memory location.

is changed to read:

MTX1	Meaning
0b0	Canonical tagging is disabled. This control has no effect on address generation.
0b1	Canonical tagging is enabled. Bits[59:56] of a 64-bit VA hold a Logical Address Tag, and all of the following apply: • Bits[59:56] are treated as 0b1111 when checking if the address is out of range. • If FEAT_PAuth is implemented, bits[59:56] are not part of the PAC field.

For more information see Logical address tagging and Memory region tagging types

Similar changes are made to the MTX0 field, the MTX1, MTX0, and MTX fields in **D24.2.195 “TCR_EL2, Translation Control Register (EL2)”**, and the MTX field in **D24.2.196 “TCR_EL3, Translation Control Register (EL3)”**.

2.39 R24537

In section **A2.2.10 “The Armv8.9 architecture extension”**, the text that reads:

FEAT_ADERR, Asynchronous Device Error Exceptions

...

FEAT_ADERR is OPTIONAL from Armv8.8.

If FEAT_ADERR is implemented, then FEAT_RASv2 is implemented.

When FEAT_AA64 and FEAT_ADERR are implemented, FEAT_AA64EL1 is implemented.

...

FEAT_ANERR, Asynchronous Normal Error Exceptions

...

FEAT_ANERR is OPTIONAL from Armv8.8.

If FEAT_ANERR is implemented, then FEAT_RASv2 is implemented.

When FEAT_AA64 and FEAT_ANERR are implemented, FEAT_AA64EL1 is implemented.

...

is changed to read:

FEAT_ADERR, Asynchronous Device Error Exceptions

...

FEAT_ADERR is OPTIONAL from Armv8.8.

...

FEAT_ANERR, Asynchronous Normal Error Exceptions

...

FEAT_ANERR is OPTIONAL from Armv8.8.

2.40 C24538

In section **D6.4.5 “External mode and MPAM”**, under the rule R_{SFTXK} , the text that reads:

If FEAT_MPAM is implemented, FEAT_TRBE_MPAM is not implemented, and the Trace Buffer Unit is using External mode, then accesses made by the Trace Buffer Unit to the trace buffer use the default MPAM values.

is changed to read:

If FEAT_MPAM is implemented, FEAT_TRBE_MPAM is not implemented, and the Trace Buffer Unit is using External mode, then accesses made by the Trace Buffer Unit to the trace buffer use the Default physical PARTID and Default PMG values, and the PARTID space corresponding to the trace physical address space defined by TRBMAR_EL1.PAS, regardless of the value of MPAMn_ELx.MPAMEN.

Under the rule R_{GJFRD} , the text that reads:

If FEAT_MPAM and FEAT_TRBE_MPAM are implemented and the Trace Buffer Unit is using External mode, then accesses made by the Trace Buffer Unit to the trace buffer use one of the following:

- The default MPAM values if TRBMPAM_EL1.EN is 0.
- If TRBMPAM_EL1.EN is 1, then the TRBMPAM_EL1.{PARTID, PMG, MPAM_SP}.

is changed to read:

If FEAT_MPAM and FEAT_TRBE_MPAM are implemented and the Trace Buffer Unit is using External mode, then accesses made by the Trace Buffer Unit to the trace buffer use one of the following:

- If TRBMPAM_EL1.EN is 0, the Default physical PARTID and Default PMG values, and the PARTID space corresponding to the trace physical address space defined by TRBMAR_EL1.PAS, regardless of the value of MPAMn_ELx.MPAMEN.
- If TRBMPAM_EL1.EN is 1, the TRBMPAM_EL1.{PARTID, PMG, MPAM_SP} MPAM values.

The following new text is added to section **D21.6 “Effect of access type on MPAM information”**:

D21.6.7 Accesses by Trace Buffer Extension

I_{x0001}

A PE with FEAT_TRBE can be configured to record trace data into a trace buffer in memory. The MPAM information used for these accesses depends on the operating mode of the Trace Buffer Unit. For more information, see:

- Section D6.3.8 “Self-hosted mode and MPAM”
- Section D6.4.5 “External mode and MPAM”

2.41 D24547

In section **J1.1.3 “aarch64/functions”**, in the pseudocode function `AArch64.CheckTag()`, the pseudocode that reads:

```
boolean AArch64.CheckTag(...)
...
constant integer granules      = Max(size DIV TAG_GRANULE, 1);

for i = 0 to granules - 1
    case memaddrdesc.memattrs.tags of
        when MemTag_AllocationTagged
            if !AArch64.AllocationTagCheck(memaddrdesc, accdesc, ltag) then
                return FALSE;
        when MemTag_CanonicalTagged
            if !AArch64.CanonicalTagCheck(memaddrdesc, ltag) then
                return FALSE;
    ...

    memaddrdesc.paddress.address = memaddrdesc.paddress.address + TAG_GRANULE;
    memaddrdesc.vaddress        = memaddrdesc.vaddress + TAG_GRANULE;

return TRUE;
```

is changed to read:

```
FaultRecord AArch64.CheckTag(...)
    AddressDescriptor memaddrdesc = memaddrdesc in;
    // NoFault() will set fault.write to FALSE to accesses that perform both a read
    and a write.
    FaultRecord fault              = NoFault(accdesc, memaddrdesc.vaddress);
    constant integer granules      = Max(size DIVRM TAG GRANULE, 1);
    constant boolean forcesync     = accdesc.nonfault || (accdesc.firstfault && !
accdesc.first);
    constant TCFTYPE tcf          = AArch64.EffectiveTCF(accdesc.el, accdesc.read);

    for i = 0 to granules - 1 do
        case memaddrdesc.memattrs.tags of
            when MemTag_AllocationTagged
                if !AArch64.AllocationTagCheck(memaddrdesc, accdesc, ltag) then
                    if tcf == TCFTYPE_Sync || forcesync then
                        fault.statuscode = Fault_TagCheck;
                        return fault;
                    elsif tcf == TCFTYPE_Async then
                        AArch64.ReportTagCheckFault(accdesc.el,
memaddrdesc.vaddress<55>);
                        return fault;
                else
                    // Tag Check Faults have no effect on the PE.
            when MemTag_CanonicalTagged
                if !AArch64.CanonicalTagCheck(memaddrdesc, ltag) then
                    if tcf == TCFTYPE_Sync || forcesync then
                        fault.statuscode = Fault_TagCheck;
                        return fault;
                    elsif tcf == TCFTYPE_Async then
                        AArch64.ReportTagCheckFault(accdesc.el,
memaddrdesc.vaddress<55>);
                        return fault;
                else
                    // Tag Check Faults have no effect on the PE.
        ...

        memaddrdesc.paddress.address = memaddrdesc.paddress.address + TAG_GRANULE;
        memaddrdesc.vaddress          = memaddrdesc.vaddress + TAG_GRANULE;
```

```
return fault;
```

In the same section, in the pseudocode functions MemStore64BWithRet() and MemLoad64B(), the code that reads:

```
if accdesc.tagchecked then
    constant bits(4) ltag = AArch64.LogicalAddressTag(address);
    if !AArch64.CheckTag(memaddrdesc, accdesc, size, ltag) then
        AArch64.TagCheckFault(address, accdesc, accdesc.write);
```

is changed to read:

```
if accdesc.tagchecked then
    constant bits(4) ltag = AArch64.LogicalAddressTag(address);
    constant FaultRecord fault = AArch64.CheckTag(memaddrdesc, accdesc, size, ltag);
    if fault.statuscode != Fault None then
        AArch64.RaiseTagCheckFault(fault);
```

A similar change is made in the pseudocode functions AArch64.CheckTag(), MemSingleNF[], AArch64.MemSingleRead(), and AArch64.MemSingleWrite().

2.42 D24553

In section **C6.2.329 “RCWSET, RCWSETA, RCWSETAL, RCWSETL”**, the description of the instruction that reads:

This instruction atomically loads a 64-bit doubleword from memory, performs a bitwise OR with the complement of the value held in a register on it, and conditionally stores the result back to memory.

is changed to read:

This instruction atomically loads a 64-bit doubleword from memory, performs a bitwise OR with the value held in a register on it, and conditionally stores the result back to memory.

The equivalent change is made in section **C6.2.331 “RCWSSET, RCWSSETA, RCWSSETAL, RCWSSETL”**.

2.43 D24555

In section **D24.9.16 “VDISR_EL2, Virtual Deferred Interrupt Status Register (EL2)”**, the text that reads:

Records that a virtual SError exception has been consumed by an ESB instruction executed at EL1.

is changed to read:

Records that a virtual SError exception has been consumed by an ESB instruction executed at EL1 or EL0.

In section **D24.9.17 “VDisr_EL3, Virtual Deferred Interrupt Status Register (EL3)”**, the text that reads:

Records that a virtual SError exception has been consumed by an ESB instruction executed at EL2 or EL1 when the Effective value of SCR_EL3.DSE is 1.

is changed to read:

Records that a virtual SError exception has been consumed by an ESB instruction executed at EL2, EL1, or EL0 when the Effective value of SCR_EL3.DSE is 1.

2.44 R24557

In section **D17.7.4 “Restrictions on the current write pointer”**, the text that reads:

This section describes the software rules on setting the current write pointer, PMBPTR_EL1.

If these rules are not followed, the value returned for a direct read of PMBPTR_EL1 is **UNKNOWN**, the behavior is **UNPREDICTABLE**, and the PE might do any of the following at any point after profiling is enabled:

- ...

When profiling becomes enabled, all the following must be true:

- The current write pointer must be at least one sample record below the write limit pointer. That is:

```
UInt(PMBPTR_EL1.PTR) <= UInt(PMBLIMITR_EL1.LIMIT:Zeros(12)) - 2^PMSIDR_EL1.MaxSize.
```

- PMBPTR_EL1.PTR[63:56] must equal PMBLIMITR_EL1.LIMIT[63:56].

is changed to read:

This section describes the software rules on setting the current write pointer, PMBPTR_EL1.

If these rules are not followed, then the behavior is **UNPREDICTABLE**, and the PE might do any of the following at any point while the Profiling Buffer is enabled, profiling is not stopped, and Discard mode is not enabled:

- PMBPTR_EL1 becomes **UNKNOWN**.
- ...

While the Profiling Buffer is enabled, profiling is not stopped, and Discard mode is not enabled, all of the following must be true:

- The current write pointer must be at least one sample record below the write limit pointer. That is:

```
UInt(PMBPTR_EL1.PTR) <= UInt(PMBLIMITR_EL1.LIMIT:Zeros(12)) - 2^PMSIDR_EL1.MaxSize.
```

- PMBPTR_EL1.PTR[63:56] must equal PMBLIMITR_EL1.LIMIT[63:56], regardless of the value of the applicable TBI bit.

When the Profiling Buffer is enabled, profiling is not stopped, and Discard mode is not enabled, the PE might ignore a direct write to any of the following Profiling Buffer registers, other than a direct write to PMBLIMITR_EL1 that clears PMBLIMITR_EL1.E from 1 to 0:

- The current write pointer, PMBPTR_EL1.
- The Limit pointer, PMBLIMITR_EL1.
- PMBSR_EL1.

Equivalent relaxations are added to the descriptions of the following registers:

- **D24.7.2 “PMBLIMITR_EL1, Profiling Buffer Limit Address Register”.**
- **D24.7.4 “PMBPTR_EL1, Profiling Buffer Write Pointer Register”.**
- **D24.7.5 “PMBSR_EL1, Profiling Buffer Status/syndrome Register (EL1)”.**

2.45 C24563

In section **C6.2.51 “CAS, CASA, CASAL, CASL”**, the text that reads:

For a CAS or CASA instruction, when <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, this signals to the memory system that an additional subsequent CAS, CASA, CASAL, or CASL access to the specified location is likely to occur in the near future. The memory system can respond by taking actions that are expected to enable the subsequent CAS, CASA, CASAL, or CASL access to succeed when it does occur.

is changed to read:

For a CAS or CASA instruction, when <Ws> or <Xs> specify the same register as <Wt> or <Xt>, this signals to the memory system that an additional subsequent CAS, CASA, CASAL, or CASL access to the specified location is expected to occur in the near future in the absence of taking an exception or exceptional software conditions. The memory system can respond by taking actions that are expected to enable the subsequent CAS, CASA, CASAL, or CASL access to succeed when it does occur.

Note:

For performance reasons, software is advised to not use a CAS where <Ws> or <Xs> specify the same register as <Wt> or <Xt> in a routine which includes a subsequent CAS for any of the following purposes:

- To load the value that will be compared to conditionally determine whether or not to execute the subsequent CAS.
- As a form of prefetch executed between the instruction loading the value that will be compared and the subsequent compare instruction which will conditionally determine whether or not to execute the subsequent CAS.

This guidance does not apply in cases where the compare between the CAS, where <Ws> or <Xs> specify the same register as <Wt> or <Xt>, and the subsequent CAS is intended to detect an exceptional software condition such that the result of the compare does not prevent the subsequent CAS from executing under normal operating conditions.

The equivalent changes are made in the following sections:

- **C6.2.52 CASB, CASAB, CASALB, CASLB.**
- **C6.2.53 CASH, CASAH, CASALH, CASLH.**
- **C6.2.54 CASP, CASPA, CASPAL, CASPL.**

2.46 D24566

FEAT_EPAC is deprecated in all future versions of Arm® Architecture Reference Manual for A-profile architecture.

2.47 D24595

In section **D1.4.6.2 “Delegated SErrors”**, the rule that reads:

R_{TNWJT}

Setting the Effective value of the SCR_EL3.EnDSE control bit to 1 enables the delegated SError exception.

is changed to read:

R_{TNWJT}

If the Effective value of SCR_EL3.{EA, TMEA} is not {0, 0}, then setting the Effective value of the SCR_EL3.EnDSE control bit to 1 enables the delegated SError exception.

If the Effective value of SCR_EL3.{EA, TMEA} is {0, 0}, or the Effective value of SCR_EL3.EnDSE is 0, then delegated SError exceptions are disabled.

In section **D24.2.169 “SCR_EL3, Secure Configuration Register”**, the following text is added to bit field ‘EnDSE’:

This field is ignored by the PE and treated as zero if SCR_EL3.EA is 0 and the Effective value of SCR_EL3.TMEA is 0.

2.48 D24642

In section **D24.7.9 “PMSCR_EL2, Statistical Profiling Control Register (EL2)”**, the description of field 'PA, bit[4]' that reads:

If EL2 is not implemented or EL2 is disabled in the owning Security state, then the Effective value of PMSCR_EL1.PA is 1.

is changed to read

If EL2 is not implemented or EL2 is disabled in the owning Security state, then the Effective value of this field is 1.

2.49 D24658

In section **J1.3.1 shared/debug**, in the pseudocode function `ExitDebugState()`, the code that reads:

```
...
    if IsFeatureImplemented(FEAT_BRBE) then
        BRBEDebugStateExit(new_pc);
...
```

is changed to read:

```
...
    if IsFeatureImplemented(FEAT_BRBE) then
        BRBEDebugStateExit(AArch64.BranchAddr(new_pc, PSTATE.EL));
...
```

2.50 D24660

In section **D2.8.7.2 “Reserved DBGBCR<n>_EL1.{SSCE, SSC, HMC, PMC} values”**, the table that reads:

HMC, SSCE, SSC, and PMC combination	Reserved
All combinations with SSCE set to 1.	When FEAT_RME is not implemented.
All combinations with HMC set to 0, SSCE set to 0, and SSC set to 0b01 or 0b10.	When Secure state is not implemented.
All combinations with SSCE set to 0 and SSC set to 0b01 or 0b10, except for the combination with HMC set to 1, SSCE set to 0, SSC set to 0b01, and PMC set to 0b00.	When EL3 is not implemented and EL2 is implemented.
Any combination with SSCE set to 0 where HMC or SSC is nonzero, except for the combination with HMC set to 1, SSCE set to 0, SSC set to 0b01, and PMC set to 0b00, or combinations when SSC is set to 0b11.	When both of EL2 and EL3 are not implemented.

HMC, SSCE, SSC, and PMC combination	Reserved
The combination with HMC set to 1, SSCE set to 0, SSC set to 0b11, and PMC set to 0b00.	When EL2 is not implemented.
The combinations with SSC set to 0b11 except the combination with HMC set to 1, SSCE set to 0, SSC set to 0b11 and PMC set to 0b00.	When Secure EL2 is not implemented.
The combination with HMC set to 1, SSCE set to 0, SSC set to 0b01 and PMC set to 0b00.	When Secure EL2 is not implemented.
Combinations not included in Table D2-8.	Always.

is changed to read:

HMC, SSCE, SSC, and PMC combination	Reserved
Any combination when HMC is 0, SSCE is 0, and SSC is 0b10, except the combination when HMC is 1, SSCE is 0, SSC is 0b10, and PMC is 0b00.	When Secure state is not implemented.
Any combination when HMC is 0, SSCE is 0, and SSC is 0b01, and the combination when HMC is 1, SSCE is 0, SSC is 0b10, and PMC is 0b00.	When FEAT_RME and Secure state are not implemented.
Any combination when SSCE is 0 and SSC is either 0b01 or 0b10.	When EL3 is not implemented and EL2 is implemented.
Any combination when SSCE is 0 and either HMC is 1 or SSC is not 0b00.	When both EL3 and EL2 are not implemented.
The combination when HMC is 1, SSCE is 0, SSC is 0b11, and PMC is 0b00.	When EL2 is not implemented.
Any combination when SSC is 0b11, except the combination when HMC is 1, SSCE is 0, SSC is 0b11, and PMC is 0b00.	When Secure EL2 is not implemented.
The combination when HMC is 1, SSCE is 0, SSC is 0b01 and PMC is 0b00.	When EL3 is not implemented or Secure EL2 is not implemented.
Combinations not included in Table D2-8.	Always.

The following text is added after the table:

When FEAT_RME is not implemented, DBGBCR<n>_EL1.SSCE is **RES0**.

Equivalent corrections are made to the table in section **D2.9.7.2 “Reserved DBGWCR<n>_EL1. {SSCE, SSC, HMC, PAC} values”**.

In section **G2.8.6.3.2 “Reserved DBGBCR<n>.{SSC, HMC, PMC} values”**, the table that reads:

HMC, SSC, and PMC combination	Reserved
All combinations with SSC set to 0b01 or 0b10.	When EL3 is not implemented and EL2 is implemented and EL2 is implemented.
Any combination where HMC or SSC is nonzero.	When both of EL2 and EL3 are not implemented.
The combinations with SSC set to 0b11 and the combination with HMC set to 1, SSC set to 0b01 and PMC set to 0b00.	When EL2 is not implemented and EL3 is implemented.
Any combination where HMC or SSC is nonzero.	When RME is not implemented.
The combination with HMC set to 1, SSC set to 0b01 and PMC set to 0b00, and the combinations with SSC set to 1, SSC set to 0b11 and PMC set to 0b00.	When both of EL2 and EL3 are implemented, but Secure EL2 is not implemented.
Combinations not included in Table G2-10.	Always.

is changed to read:

HMC, SSC, and PMC combination	Reserved
All combinations when SSC is either 0b01 or 0b10.	When Secure state is not implemented.
Any combination when HMC is 1 or SSC is not 0b00.	When both EL3 and EL2 are not implemented.
The combination when HMC is 1, SSC is 0b11, and PMC is 0b00.	When EL2 is not implemented.
Any combinations when SSC is 0b11, and PMC is 0b01 or 0b11.	When Secure EL2 is not implemented.
The combination when HMC is 1, SSC is 0b01, and PMC is 0b00.	When EL3 is not implemented or Secure EL2 is not implemented.
Combinations not included in Table G2-10.	Always.

In section **G2.9.6.1 “Reserved DBGWCR<n>.{SSC, HMC, PAC} values”**, the table that reads:

HMC, SSC, and PAC combination	Reserved
All combinations with SSC set to 0b01 or 0b10.	When EL3 is not implemented and EL2 is implemented.
Any combination where HMC or SSC is nonzero.	When both of EL2 and EL3 are not implemented.
The combination with HMC set to 1, SSC set to 0b11, and PAC set to 0b00.	When EL2 is not implemented.
The combinations with SSC set to 0b11 and PAC set to 0b01 or 0b11.	When Secure EL2 is not implemented.
The combination with HMC set to 1, SSC set to 0b01 and PAC set to 0b00.	When Secure EL2 is not implemented.
Combinations not included in Table G2-15.	Always.

is changed to read:

HMC, SSC, and PAC combination	Reserved
All combinations when SSC is either 0b01 or 0b10.	When Secure state is not implemented.
Any combination when HMC is 1 or SSC is not 0b00.	When both EL3 and EL2 are not implemented.
The combination when HMC is 1, SSC is 0b11, and PAC is 0b00.	When EL2 is not implemented.
Any combination when SSC is 0b11, and PAC is either 0b01 or 0b11.	When Secure EL2 is not implemented.
The combination when HMC is 1, SSC is 0b01, and PAC is 0b00.	When EL3 is not implemented or Secure EL2 is not implemented.
All combinations not included in Table G2-15.	Always.

In section **J1.1.3 “aarch64/functions”**, in the function CheckValidStateMatch(), the code segment that reads:

```
// Values that are not allocated in any architecture version
case hmc:ssc:pxc of
  when '0 0 11 10' reserved = TRUE;
  when '0 0 1x xx' reserved = !HaveSecureState();
  when '1 0 00 x0' reserved = TRUE;
  when '1 0 01 10' reserved = TRUE;
  when '1 0 1x 10' reserved = TRUE;
  when 'x 1 xx xx' reserved = ssc != '01' || (hmc:pxc) IN {'000', '110'};
  otherwise      reserved = FALSE;

...

// Secure EL2 is not implemented
if !IsFeatureImplemented(FEAT_SEL2) && (hmc:ssc:pxc) IN
{'01100', '10100', 'x11x1'} then
  reserved = TRUE;
```


is changed to read:

```
// Values that are not allocated in any architecture version
case hmc:ssce:ssc:pxc of
  when '0 0 11 10' reserved = TRUE;
  when '0 1 01 00' reserved = TRUE;
  when '1 0 00 x0' reserved = TRUE;
  when '1 0 01 10' reserved = TRUE;
  when '1 0 1x 10' reserved = TRUE;
  when '1 1 01 10' reserved = TRUE;
  when 'x 1 x0 xx' reserved = TRUE;
  when 'x 1 11 xx' reserved = TRUE;

  // Secure state not implemented
  if !IsFeatureImplemented(FEAT_Secure) && hmc == '0' && ssce == '0' && ssc IN
{'01', '10'} && hmc:ssce:ssc:pxc != '101000' then
    reserved = TRUE;

  // FEAT_RME and Secure state not implemented
  if !IsFeatureImplemented(FEAT_RME) && !IsFeatureImplemented(FEAT_Secure) &&
hmc:ssc:pxc == '11000' then
    reserved = TRUE;

  ...

  // Secure EL2 is not implemented
  if !IsFeatureImplemented(FEAT_SEL2) && ssc == '11' && hmc:ssce:ssc:pxc !=
'101100' then
    reserved = TRUE;

  // Secure EL2 is not implemented or EL3 is not implemented
  if (!IsFeatureImplemented(FEAT_SEL2) || !HaveEL(EL3)) && hmc:ssce:ssc:pxc ==
'100100' then
    reserved = TRUE;
```

In addition, the following AArch32 changes are made.

In section **G2.8.3 “Execution conditions for which a breakpoint generates Breakpoint exceptions”**, in Table G2-10, the entries that indicate matches in Secure EL2 using AArch32 are changed to “-” because such matches are not possible.

In section **G2.9.3 “Execution conditions for which a watchpoint generates Watchpoint exceptions”**, in Table G2-15:

- The entries that indicate matches in Secure EL2 using AArch32 are changed to “-”.
- The “Implementation” columns and their corresponding footnotes are removed, because the information is instead provided in section **G2.9.6.1 “Reserved DBGWCR<n>.{SSC, HMC, PAC} values”**.

2.51 R24661

In section **I5.3.35 “PMIIDR, Performance Monitors Implementation Identification Register”**, the text that reads:

ProductID, bits [31:20]

Part number, bits [11:0]. The part number is selected by the designer of the component.

Matches the PMPIDR1.PART_1, PMPIDR0.PART_0 fields if present.

This field has an **IMPLEMENTATION DEFINED** value.

Access to this field is RO.

Variant, bits [19:16]

Component major revision.

Defines either a variant of the component defined by PMIIDR.ProductID, or the major revision of the component.

When defining a major revision, PMIIDR.Variant and PMIIDR.Revision together form the revision number of the component, with this field being the most significant part.

When a component is changed, PMIIDR.Variant or PMIIDR.Revision is increased to ensure that software can differentiate between different revisions of the component. If this field is increased, PMIIDR.Revision should be set to 0b0000.

Matches the PMPIDR2.REVISION field, if present.

This field has an **IMPLEMENTATION DEFINED** value.

Access to this field is RO.

Revision, bits [15:12]

Component minor revision.

PMIIDR.Variant and PMIIDR.Revision together form the revision number of the component, with this field being the least significant part.

When a component is changed, PMIIDR.Variant or PMIIDR.Revision is increased to ensure that software can differentiate between different revisions of the component. If PMIIDR.Variant field is increased, this field should be set to 0b0000, otherwise the value in this field should be increased.

Matches the PMPIDR3.REVAND field, if present.

This field has an **IMPLEMENTATION DEFINED** value.

Access to this field is RO.

is changed to read:

ProductID, bits [31:20]

Part number, bits [11:0]. The part number is selected by the designer of the component.

Arm recommends this field matches the part number represented in the Peripheral ID registers PMPIDRn, if those registers are present.

This field has an **IMPLEMENTATION DEFINED** value.

Access to this field is RO.

Variant, bits [19:16]

Component major revision.

Defines either a variant of the component defined by PMIIDR.ProductID, or the major revision of the component.

When defining a major revision, PMIIDR.Variant and PMIIDR.Revision together form the revision number of the component, with this field being the most significant part.

When a component is changed, PMIIDR.Variant or PMIIDR.Revision is increased to ensure that software can differentiate between different revisions of the component. If this field is increased, PMIIDR.Revision should be set to 0b0000.

Arm recommends this field matches the PMPIDR2.REVISION field, if present.

This field has an **IMPLEMENTATION DEFINED** value.

Access to this field is RO.

Revision, bits [15:12]

Component minor revision.

PMIIDR.Variant and PMIIDR.Revision together form the revision number of the component, with this field being the least significant part.

When a component is changed, PMIIDR.Variant or PMIIDR.Revision is increased to ensure that software can differentiate between different revisions of the component. If PMIIDR.Variant field is increased, this field should be set to 0b0000, otherwise the value in this field should be increased.

Arm recommends this field matches the PMPIDR3.REVAND field, if present.

This field has an **IMPLEMENTATION DEFINED** value.

Access to this field is RO.

In section **I5.5.24 “AMIIDR, Activity Monitors Implementation Identification Register”**, the text that reads:

ProductID, bits [31:20]

This field is an AMU part identifier.

This field has an **IMPLEMENTATION DEFINED** value.

If AMPIDR0 is implemented, AMPIDR0.PART_0 matches bits [27:20] of this field.

If AMPIDR1 is implemented, AMPIDR1.PART_1 matches bits [31:28] of this field.

Access to this field is RO.

Variant, bits [19:16]

This field distinguishes product variants or major revisions of the product.

This field has an **IMPLEMENTATION DEFINED** value.

If AMPIDR2 is implemented, AMPIDR2.REVISION matches AMIIDR.Variant.

Access to this field is RO.

Revision, bits [15:12]

This field distinguishes minor revisions of the product.

This field has an **IMPLEMENTATION DEFINED** value.

If AMPIDR3 is implemented, AMPIDR3.REVAND matches AMIIDR.Revision.

Access to this field is RO.

is changed to read:

ProductID, bits [31:20]

Part number of the AMU, bits [11:0]. The part number is selected by the designer of the component.

Arm recommends this field matches the part number represented in the Peripheral ID registers AMPIDRn, if those registers are present.

This field has an **IMPLEMENTATION DEFINED** value.

Access to this field is RO.

Variant, bits [19:16]

Component major revision.

Defines either a variant of the component defined by AMIIDR.ProductID, or the major revision of the component.

When defining a major revision, AMIIDR.Variant and AMIIDR.Revision together form the revision number of the component, with this field being the most significant part.

When a component is changed, AMIIDR.Variant or AMIIDR.Revision is increased to ensure that software can differentiate between different revisions of the component. If this field is increased, AMIIDR.Revision should be set to 0b0000.

Arm recommends this field matches the AMPIDR2.REVISION field, if present.

This field has an **IMPLEMENTATION DEFINED** value.

Access to this field is RO.

Revision, bits [15:12]

Component minor revision.

AMIIDR.Variant and AMIIDR.Revision together form the revision number of the component, with this field being the least significant part.

When a component is changed, AMIIDR.Variant or AMIIDR.Revision is increased to ensure that software can differentiate between different revisions of the component. If AMIIDR.Variant field is increased, this field should be set to 0b0000, otherwise the value in this field should be increased.

Arm recommends this field matches the AMPIDR3.REVAND field, if present.

This field has an **IMPLEMENTATION DEFINED** value.

Access to this field is RO.

2.52 D24669

In section **J1.1.3 “aarch64/functions”**, in the pseudocode function IsAPIAKeyEnabled(), the code that reads:

```
if IsFeatureImplemented(FEAT_PAuth_EnhCtl) then
    Enable = Enable EOR (if IsEL1Regime then SCTLR2_EL1.EnIA2 else
SCTLR2_EL2.EnIA2);
```

is changed to read:

```
if IsFeatureImplemented(FEAT_PAuth_EnhCtl) then
    bit enia2;
    if IsEL1Regime then
        enia2 = if IsSCTLR2EL1Enabled() then SCTLR2_EL1.EnIA2 else '0';
```

```
else
    enia2 = if IsSCTLR2EL2Enabled() then SCTLR2_EL2.EnIA2 else '0';
Enable = Enable EOR enia2;
```

Similar changes are made in the pseudocode functions `IsAPIBKeyEnabled()`, `IsAPDAKeyEnabled()`, and `IsAPDBKeyEnabled()`.

2.53 C24673

In section **I5.3.12 “PMCID1SR, CONTEXTIDR_EL1 Sample Register”**, under the heading ‘Purpose’, the text that reads:

Contains the sampled value of `CONTEXTIDR_EL1`, captured on reading `PMPCSR[31:0]`.

is changed to read:

PC Sample-based Profiling Extension register that contains the sampled value of `CONTEXTIDR_EL1`, captured on reading `PMPCSR[31:0]`.

Similar changes are made to the following sections:

- **I5.3.13 “PMCID2SR, CONTEXTIDR_EL2 Sample Register”**.
- **I5.3.48 “PMPCSR, Program Counter Sample Register”**.
- **I5.3.56 “PMVIDSR, VMID Sample Register”**.
- **I5.3.3 “PMCCIDSR, CONTEXTIDR_ELx Sample Register”**.
- **I5.3.57 “PMVCIDSR, CONTEXTIDR_EL1 and VMID Sample Register”**.

2.54 R24675

In section **H9.2.37 “EDPIDR0, External Debug Peripheral Identification Register 0”**, the text that reads:

`PART_0`, bits [7:0]

Part number, least significant byte.

is changed to read:

`PART_0`, bits [7:0]

Part number, which is selected by the designer of the component and stored as follows:

- For a component with a 12-bit part number:
 - `EDPIDR1.PART_1` contains part number bits [11:8].
 - `EDPIDR0.PART_0` contains part number bits [7:0].

- For a component with a 16-bit part number:
 - EDPIDR1.PART_1 contains part number bits [15:12].
 - EDPIDR0.PART_0 contains part number bits [11:4].
 - EDPIDR2.REVISION contains part number bits [3:0].

When a 12-bit part number is used, PIDR2.REVISION indicates revision information.

The choice of using a 12-bit part number or 16-bit part number is specific to the designer of the component.

This field has an **IMPLEMENTATION DEFINED** value.

Access to this field is RO.

In section **H9.2.38 “EDPIDR1, External Debug Peripheral Identification Register 1”**, the text that reads:

PART_1, bits [3:0]

Part number, most significant nibble.

is changed to read:

PART_1, bits [3:0]

Part number, which is selected by the designer of the component and stored as follows:

- For a component with a 12-bit part number:
 - EDPIDR1.PART_1 contains part number bits [11:8].
 - EDPIDR0.PART_0 contains part number bits [7:0].
- For a component with a 16-bit part number:
 - EDPIDR1.PART_1 contains part number bits [15:12].
 - EDPIDR0.PART_0 contains part number bits [11:4].
 - EDPIDR2.REVISION contains part number bits [3:0].

When a 12-bit part number is used, PIDR2.REVISION indicates revision information.

The choice of using a 12-bit part number or 16-bit part number is specific to the designer of the component.

This field has an **IMPLEMENTATION DEFINED** value.

Access to this field is RO.

In section **H9.2.39 “EDPIDR2, External Debug Peripheral Identification Register 2”**, the text that reads:

REVISION, bits [7:4]

Part major revision. Parts can also use this field to extend Part number to 16-bits.

is changed to read:

REVISION, bits [7:4]

Indicates either the revision of the component, or a portion of the part number of the component.

Where the component has a single 4-bit revision number, the revision number is an incremental value starting at zero for the first revision of the component.

Where the component has separate major and minor revision numbers, the major and minor revision numbers are each incremental values starting at zero for the first revision of the component. For each minor revision of the component, the minor revision number increments monotonically. For each major revision of the component, the major revision number increments monotonically and the minor revision begins again at zero.

For a component with a 12-bit part number with a single 4-bit revision number:

- EDPIDR2.REVISION indicates the 4-bit revision number.
- EDPIDR3.REVAND indicates component modifications.

For a component with a 12-bit part number with separate major and minor revision numbers:

- EDPIDR2.REVISION indicates the 4-bit major revision number.
- EDPIDR3.REVAND indicates the 4-bit minor revision number.

For a component with a 16-bit part number:

- EDPIDR2.REVISION contains part number bits [3:0].
- EDPIDR3.REVAND indicates the 4-bit revision number.

The choice of which style of revision information is used is specific to the designer of the component, and might also be specific to each individual component with a different part number.

This field has an **IMPLEMENTATION DEFINED** value.

Access to this field is RO.

In section **H9.2.40 “EDPIDR3, External Debug Peripheral Identification Register 3”**, the text that reads:

REVAND, bits [7:4]

Part minor revision. Parts using EDPIDR2.REVISION as an extension to the Part number must use this field as a major revision number.

is changed to read:

REVAND, bits [7:4]

Indicates either the revision of the component, or whether the component has been modified.

Where the component has a single 4-bit revision number, the revision number is an incremental value starting at zero for the first revision of the component.

Where the component has separate major and minor revision numbers, the major and minor revision numbers are each incremental values starting at zero for the first revision of the component. For each minor revision of the component, the minor revision number increments monotonically. For each major revision of the component, the major revision number increments monotonically and the minor revision begins again at zero.

For a component with a 12-bit part number with a single 4-bit revision number:

- EDPIDR2.REVISION indicates the 4-bit revision number.
- EDPIDR3.REVAND indicates component modifications.

For a component with a 12-bit part number with separate major and minor revision numbers:

- EDPIDR2.REVISION indicates the 4-bit major revision number.
- EDPIDR3.REVAND indicates the 4-bit minor revision number.

For a component with a 16-bit part number:

- EDPIDR2.REVISION contains part number bits [3:0].
- EDPIDR3.REVAND indicates the 4-bit revision number.

The choice of which style of revision information is used is specific to the designer of the component, and might also be specific to each individual component with a different part number.

Where REVAND indicates component modifications, this indicates modifications such as errata fixes or metal fixes after implementation. Usually this value would be zero unless a modification has been performed. If the field is required for indicating component modifications, Arm recommends that component designers ensure that it can be changed by a metal fix, for example by driving it from registers that reset to zero.

PIDR3.CMOD might also indicate component modifications.

This field has an **IMPLEMENTATION DEFINED** value.

Access to this field is RO.

CMOD, bits [3:0]

Indicates whether the component has been modified from its original behavior. Examples of modifications include errata fixes or metal fixes after implementation. Usually this value would be

zero unless a modification has been performed. If the field is required for indicating component modifications, Arm recommends that component designers ensure that it can be changed by a metal fix, for example by driving it from registers that reset to zero.

A value of 0b0000 means the component is not modified from the original design.

Any other value means the component has been modified in an **IMPLEMENTATION DEFINED** way.

For any two components with the same Unique Component Identifier:

- If the value of the CMOD fields of both components equals zero, the components are identical.
- If the CMOD fields of both components have the same nonzero value, it does not necessarily mean that they have the same modifications.
- If the value of the CMOD field of either of the two components is nonzero, they might not be identical, even though they have the same Unique Component Identifier.

EDPIDR3.REVAND might also indicate component modifications.

This field has an **IMPLEMENTATION DEFINED** value.

Access to this field is RO.

The equivalent changes are made to the following sections:

- **H9.3 “TRCPIDRn, Trace Peripheral Identification Register n”.**
- **H9.4 “TRBPIDRn, Peripheral Identification Register n”.**
- **H9.5 “CTIPIDRn, CTI Peripheral Identification Register n”.**
- **I5.3 “PMPIDRn, Performance Monitors Peripheral Identification Register n”.**
- **I5.5 “AMPIDRn, Activity Monitors Peripheral Identification Register n”.**

The equivalent changes are also made in the *Arm Reliability, Availability, and Serviceability (RAS) System Architecture for A-profile architecture* (IHI 0100), Issue A.b.

2.55 R24687

In section **A2.3.7 “The Armv9.6 architecture extension”**, in feature FEAT_FPRCVT, the text that reads:

This feature is OPTIONAL in Armv9.5 implementations. When FEAT_FP is implemented, this feature is mandatory in Armv9.6 implementations.

is changed to read:

This feature is OPTIONAL in Armv9.5 implementations. When FEAT_FP is implemented, this feature is mandatory in Armv9.7 implementations.

In the same section, in feature FEAT_SVE2p2, the text that reads:

This feature is OPTIONAL in Armv9.5 implementations. When FEAT_SVE2 is implemented, this feature is mandatory in Armv9.6 implementations.

is changed to read:

This feature is OPTIONAL in Armv9.5 implementations.

Similarly, in feature FEAT_SME2p2, the text that reads:

This feature is OPTIONAL in Armv9.5 implementations. When FEAT_SME is implemented, this feature is mandatory in Armv9.6 implementations.

is changed to read:

This feature is OPTIONAL in Armv9.5 implementations.

2.56 D24695

In section **J1.1.3.444 “GPTTLBIMatch”**, the code that reads:

```
case tlbi.op of
    when TLBIOp_RPA
        match = (UInt(tlbi.address<55:0>) <= UInt(entry_end_address<55:0>) &&
                UInt(tlbi.end_address<55:0>) > UInt(entry_start_address<55:0>))
        &&
            (tlbi.level == TLBILevel_Any || gpt_entry.level == 1));
```

is changed to read:

```
case tlbi.op of
    when TLBIOp_RPA
        match = (UInt(tlbi.address<55:0>) <= UInt(entry_end_address<55:0>) &&
                UInt(tlbi.end_address<55:0>) > UInt(entry_start_address<55:0>))
        &&
            (tlbi.level == TLBILevel_Any || !gpt_entry.istable));
```

In section **J1.3.5.46 “GPTEntry”**, the code that reads:

```
type GPTEntry is (
    bits(4) gpi,                // GPI value for this region
    AddressSize size,           // Region size
    AddressSize contig_size,    // Contiguous region size
    integer level,              // Level of GPT lookup
    bits(MAX_PASIZE) pa         // PA uniquely identifying the GPT entry
)
```

is changed to read:

```
type GPTEntry is (
    bits(4) gpi,                // GPI value for this region
```

```

AddressSize    size,          // Region size
AddressSize    contig_size,   // Contiguous region size
boolean        istable,       // Flag a table entry
integer        level,         // Level of GPT lookup
bits(MAX_PASIZE) pa           // PA uniquely identifying the GPT entry
)

```

2.57 D24696

In section **J1.1.3.435 “AArch64.TLBI_RPA”**, the code that reads:

```

bits(52) BaseADDR = Zeros(52);
case GPCCR_EL3.PGS of
  when '00' BaseADDR<51:12> = Xt<39:0>;    // 4KB
  when '10' BaseADDR<51:14> = Xt<39:2>;    // 16KB
  when '01' BaseADDR<51:16> = Xt<39:4>;    // 64KB

// The calculation here automatically aligns BaseADDR to the size of
// the region specified in SIZE. However, the architecture does not
// require this alignment and if BaseADDR is not aligned to the region
// specified by SIZE then no entries are required to be invalidated.
constant integer range_pbits = range_bits;
constant bits(52) start_addr = BaseADDR AND NOT ZeroExtend(Ones(range_pbits),
52);
constant bits(52) end_addr  = start_addr + ZeroExtend(Ones(range_pbits), 52);

```

is changed to read:

```

bits(56) BaseADDR = Zeros(56);
case GPCCR_EL3.PGS of
  when '00' BaseADDR<51:12> = Xt<39:0>;    // 4KB
  when '10' BaseADDR<51:14> = Xt<39:2>;    // 16KB
  when '01' BaseADDR<51:16> = Xt<39:4>;    // 64KB

if IsFeatureImplemented(FEAT_D128) && AArch64.PAMax() == 56 then
  BaseADDR<55:52> = Xt<43:40>; // Extension to Address

// The calculation here automatically aligns BaseADDR to the size of
// the region specified in SIZE. However, the architecture does not
// require this alignment and if BaseADDR is not aligned to the region
// specified by SIZE then no entries are required to be invalidated.
constant integer range_pbits = range_bits;
constant bits(56) start_addr = BaseADDR AND NOT ZeroExtend(Ones(range_pbits),
56);
constant bits(56) end_addr  = start_addr + ZeroExtend(Ones(range_pbits), 56);

```

2.58 D24722

In section **J1.3.5 “shared/translation”**, in the pseudocode function `DecodeGPTTable()`, the code that reads:

```

GPTTable DecodeGPTTable(PGSe pgs, bits(64) gpt_entry)
...
case pgs of
  when PGS_4KB  result.address = gpt_entry<55:17>:Zeros(17);
  when PGS_16KB result.address = gpt_entry<55:15>:Zeros(15);

```

```

        when PGS_64KB result.address = gpt_entry<55:13>:Zeros(13);
        otherwise Unreachable();

    return result;

```

is changed to read:

```

GPTTable DecodeGPTTable(PGSe pgs, bits(64) gpt_entry)
...
// Descriptor bits [LOGPTSZ-PGS-2:12] are RES0
integer s;
case GPTL0Size() of
    when GPTRange_1GB    s = 30;
    when GPTRange_16GB   s = 34;
    when GPTRange_64GB   s = 36;
    when GPTRange_512GB  s = 39;
    otherwise Unreachable();

integer p;
case pgs of
    when PGS_4KB    p = 12;
    when PGS_16KB   p = 14;
    when PGS_64KB   p = 16;
    otherwise Unreachable();

constant integer align_bits = (s - p) - 1;
result.address = Align(gpt_entry[55:0], align_bits)

return result;

```

2.59 D24726

In section **D17.8.4.4 “The External abort is reported to the SPU”**, the text that reads:

- PMBSR_EL1.EC is set to one of:
 - 0b100100, stage 1 Data Abort on write to buffer.
 - 0b100101, stage 2 Data Abort on write to buffer.

is changed to read:

- PMBSR_EL1.EC is set as follows:
 - If the External abort is on a GPT fetch, 0b011110, Granule Protection Check fault on write to Profiling Buffer, other than Granule Protection Fault (GPF).
 - If the External abort is on a stage 2 translation table access, 0b100101, stage 2 Data Abort on write to buffer.
 - Otherwise, 0b100100, stage 1 Data Abort on write to buffer.

An equivalent change is made in section **D6.5.5.4 “The External abort is reported to the Trace Buffer Unit”**.

2.60 D24779

In section **D24.2.176 “SCTLR_EL2, System Control Register (EL2)”**, the text that reads:

EOE, bit [24]

When ELIsInHost(EL2) and FEAT_MixedEndELO is implemented: Endianness of data accesses at ELO.

EOE	Meaning
0b0	Explicit data accesses at ELO are little-endian.
0b1	Explicit data accesses at ELO are big-endian.

If HCR_EL2.TGE == 0b0, the field is IGNORED for all purposes other than direct reads and writes of the register.

This bit has no effect on the endianness of Explicit Memory Effects generated by unprivileged memory access instructions executed at EL2.

The reset behavior of this field is:

- On a Warm reset, this field resets to an architecturally **UNKNOWN** value.

When FEAT_BigEndELO is implemented:

Explicit data accesses at ELO are big-endian.

Reserved, **RES1**.

Otherwise:

Explicit data accesses at ELO are little-endian.

Reserved, **RES0**.

is changed to read:

EOE, bit [24]

When FEAT_MixedEndELO is implemented and the Effective value of HCR_EL2.E2H == 1:

Endianness of data accesses at ELO.

EOE	Meaning
0b0	Explicit data accesses at ELO are little-endian.
0b1	Explicit data accesses at ELO are big-endian.

If the Effective value of HCR_EL2.TGE is 0, this field is IGNORED for all purposes other than direct reads and writes of the register.

This bit has no effect on the endianness of Explicit Memory Effects generated by unprivileged memory access instructions executed at EL2.

The reset behavior of this field is:

- On a Warm reset, this field resets to an architecturally **UNKNOWN** value.

When FEAT_BigEndELO is implemented and the Effective value of HCR_EL2.E2H == 1:

Explicit data accesses at ELO are big-endian.

Reserved, **RES1**.

When FEAT_LittleEndELO is implemented and the Effective value of HCR_EL2.E2H == 1:

Explicit data accesses at ELO are little-endian.

Reserved, **RES0**.

Otherwise:

Reserved, **RES0**.

2.61 D24784

The Synchronous Exception-based Event Profiling (FEAT_SEBEP) is withdrawn from all future versions of *Arm® Architecture Reference Manual for A-profile architecture*.

2.62 C24810

In section **D7.5.10 “Data cache zero instruction”**, the text that reads:

If the memory region being zeroed is any type of Device memory, then DC ZVA generates an Alignment fault which is prioritized in the same way as other alignment faults that are determined by the memory type.

is changed to read:

If the memory region being zeroed is any type of Device memory that does not support unaligned accesses, then DC ZVA generates an Alignment fault which is prioritized in the same way as other alignment faults that are determined by the memory type.

Similarly, in section **C5.3.38 “DC ZVA, Data Cache Zero by VA”**, the text that reads:

If the memory region being zeroed is any type of Device memory, this instruction generates an Alignment fault which is prioritized in the same way as other Alignment faults that are determined by the memory type.

is changed to read:

If the memory region being zeroed is any type of Device memory that does not support unaligned accesses, this instruction generates an Alignment fault which is prioritized in the same way as other Alignment faults that are determined by the memory type.

The equivalent change is made in section **C5.3.25 “DC GZVA, Data Cache set Allocation Tags and Zero by VA”**.

In section **C5.3.30 “DC GVA, Data Cache set Allocation Tag by VA”**, the text that reads:

A DC GVA instruction to any type of Device memory is **CONSTRAINED UNPREDICTABLE** between:

- Not modifying Allocation tags at the specified addresses.
- Generating an Alignment Fault determined by the memory type.

is changed to read:

For a DC GVA instruction to any type of Device memory, it is **CONSTRAINED UNPREDICTABLE** whether it generates an Alignment Fault determined by the memory type. Allocation tags at the specified addresses are not modified.

The same change for DC GVA also applies to the DC ZGBVA and DC GBVA instructions.

2.63 C24818

In section **J1.3.3.324 “IsContiguousSVEAccess()”**, in the pseudocode function IsContiguousSVEAccess(), the code that reads:

```
boolean IsContiguousSVEAccess (AccessDescriptor accdesc)
    return (IsFeatureImplemented (FEAT_SVE) &&
            accdesc.acctype == AccessType_SVE &&
            accdesc.contiguous);
```

is changed to read:

```
boolean IsContiguousSVEAccess (AccessDescriptor accdesc)
    return (accdesc.acctype == AccessType_SVE &&
            accdesc.contiguous);
```

2.64 D24835

In section **D2.8.3.2 “Breakpoint types defined by DBGBCR<n>_EL1.{BT2, BT}”**, the text that reads:

{0b1, 0b0101}, Linked Address Mismatch breakpoint with linking enabled

{BT2, BT} == {0b1, 0b0100} is a reserved value if one of the following applies:

- The breakpoint does not support being configured as an address matching breakpoint with linking enabled.
- FEAT_BWE is implemented.

is changed to read:

{0b1, 0b0101}, Linked Address Mismatch breakpoint with linking enabled

{BT2, BT} == {0b1, 0b0101} is a reserved value if one of the following applies:

- The breakpoint does not support being configured as an address matching breakpoint with linking enabled.
- FEAT_BWE is not implemented.

2.65 E24841

In section **J1.1.3 “aarch64/functions”**, the pseudocode function FPScale() that reads:

```
bits(N) FPScale(bits(N) op, integer scale, FPCR_Type fpcr)
...
else
    result = FPRound(value * (2.0^scale), fpcr);
    FPPProcessDenorm(fptype, N, fpcr);
    return result;
```

is changed to read:

```
bits(N) FPScale(bits(N) op, integer scale, FPCR_Type fpcr)
...
else
    constant integer clamped_scale = FP ClampScale(op, scale);
    result = FPRound(value * (2.0^clamped_scale), fpcr);
    FPPProcessDenorm(fptype, N, fpcr);
    return result;

integer FP ClampScale(bits(N) op, integer scale_in)
    assert N IN {16, 32, 64};
    integer E;
    integer exp;
    case N of
        when 16
            E = 5;
            exp = UInt(op[14:10]);
        when 32
            E = 8;
            exp = UInt(op[30:23]);
        when 64
            E = 11;
            exp = UInt(op[62:52]);

    constant integer F = N - (E + 1);
    constant integer emax = (1 << E) - 1;
    constant integer min_scale = -(F + 1);
    constant integer max_scale = emax + (F + 1);
```

```
constant integer scale = Max(min_scale - exp, Min(scale_in, max_scale - exp));
return scale;
```

For consistency, in the same section, the pseudocode function `BFScale()` is also changed to use `BFClampScale()`:

```
bits(16) BFScale(bits(16) op, integer scale, FPCR_Type fpcr)
...
else
    constant integer clamped_scale = BFClampScale(op, scale);
    result = FPRoundBF(value * (2.0^clamped_scale), fpcr, rounding, fpexc);
    if fpexc then FPProcessDenorm(fptype, 32, fpcr);

    return result[31:16];

integer BFClampScale(bits(16) op, integer scale_in)
    constant integer E = 8;
    constant integer F = 7;
    constant integer exp = UInt(op[14:7]);
    constant integer emax = (1 << E) - 1;
    constant integer min_scale = -(F + 1);
    constant integer max_scale = emax + (F + 1);

    constant integer scale = Max(min_scale - exp, Min(scale_in, max_scale - exp));
    return scale;
```

2.66 R24849

In section **C6.2.323 “RCWCLR, RCWCLRA, RCWCLRAL, RCWCLRL”**, the description that reads:

This instruction atomically loads a 64-bit doubleword from memory, performs a bitwise AND with the complement of the value held in a register on it, and conditionally stores the result back to memory. Storing of the result back to memory is conditional on RCW Checks. The value initially loaded from memory is returned in the destination register. This instruction updates the condition flags based on the result of the update of memory.

is changed to read:

This instruction atomically loads a 64-bit doubleword from memory, performs a bitwise AND with the complement of the value held in a register on it, and conditionally stores the result back to memory. Storing of the result back to memory is conditional on RCW Checks. The value initially loaded from memory is returned in the destination register. If the RCW Checks fail, the architecture permits writing the value read from the location to memory. If the write is performed, the read and the write occur atomically such that no other modification of the memory location can take place between the read and the write. This instruction updates the condition flags based on the result of the RCW Checks.

In section **C6.2.327 “RCWSCLR, RCWSCLRA, RCWSCLRAL, RCWSCLRL”**, the description that reads:

This instruction atomically loads a 64-bit doubleword from memory, performs a bitwise AND with the complement of the value held in a register on it, and conditionally stores the result back to

memory. Storing of the result back to memory is conditional on RCW Checks and RCWS Checks. The value initially loaded from memory is returned in the destination register. This instruction updates the condition flags based on the result of the update of memory.

is changed to read:

This instruction atomically loads a 64-bit doubleword from memory, performs a bitwise AND with the complement of the value held in a register on it, and conditionally stores the result back to memory. Storing of the result back to memory is conditional on RCW Checks and RCWS Checks. The value initially loaded from memory is returned in the destination register. If the RCW Checks or RCWS Checks fail, the architecture permits writing the value read from the location to memory. If the write is performed, the read and the write occur atomically such that no other modification of the memory location can take place between the read and the write. This instruction updates the condition flags based on the result of the RCW Checks and RCWS Checks.

Equivalent relaxations are made for all of the RCW(S)* instructions other than the CAS variants of the instructions.

2.67 C24856

In section **D6.5.3 “Trigger event”**, the following text is added to statement I_{HHLBM} :

Figure D6-2 does not show all possible transitions. For example, software can also program TRBSR_EL1.TRG to 1 to indicate that a Detected Trigger has occurred. This might be used to start the trigger counter immediately after enabling the Trace Buffer Unit, or after restoring state due to a context switch.

2.68 R24862

In section **D1.4.4.1 “Legal exception returns from AArch64 state”**, the rule that reads:

R_{BWCfk}

On a legal exception return from an Exception level, ELx, all of the following occur:

- [...].
- After the PC is set to the value held in ELR_ELx and the contents of PSTATE are set to the values held in SPSR_ELx, ELR_ELx and SPSR_ELx become **UNKNOWN**.
- [...].

is changed to read:

R_{BWCfk}

On a legal exception return from an Exception level, ELx, all of the following occur:

- [...].
- After the PC is set to the value held in ELR_ELx and the contents of PSTATE are set to the values held in SPSR_ELx, ELR_ELx and SPSR_ELx become **UNKNOWN**.
- If the exception return is from EL2, to ELO in AArch64 state when HCR_EL2.TGE is 0, ELR_EL1 and SPSR_EL1 become **UNKNOWN**.
- [...].

2.69 D24867

In section **J1.3.5 “shared/translation”**, in the pseudocode function GPTTableDescriptorValid(), the code that reads:

```
boolean GPTTableDescriptorValid(bits(64) level_0_entry)
...
return IsZero(level_0_entry<63:52,11:4>) && IsZero(level_0_entry<(10sz-
p)-2:12>);
```

is changed to read:

```
boolean GPTTableDescriptorValid(bits(64) level_0_entry)
...
constant integer top = if DecodePPS() == 56 then 56 else 52;
return IsZero(level_0_entry<63:top,11:4>) && IsZero(level_0_entry<(10sz-
p)-2:12>);
```

2.70 D24887

In section **D24.2.48 “FAR_EL1, Fault Address Register (EL1)”**, the text that reads:

When FEAT_MOPS is implemented, the value in FAR_EL1 on a synchronous exception from any of the Memory Copy and Memory Set instructions -represents the first element that has not been copied or set, and- is determined as follows:

- ...

is changed to read:

When FEAT_MOPS is implemented, the value in FAR_EL1 on a synchronous exception from any of the Memory Copy and Memory Set instructions is determined as follows:

- ...

The equivalent correction is made in the following sections:

- **D24.2.49 “FAR_EL2, Fault Address Register (EL2)”**.
- **D24.2.50 “FAR_EL3, Fault Address Register (EL3)”**.

2.71 C24897

In section **C5.5.29 TLBI RVAEE1, TLBI RVAEE1NXS, TLB Range Invalidate by VA, All ASID, EL1**, in the EL1 accessibility code, the code that reads:

```
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TTLB == '1' && ... then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && ... && HFGITR_EL2.TLBIRVAAE1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && HCR_EL2.FB == '1' && !(IsFeatureImplemented(FEAT_TLBIID) &&
        IsHCRXEL2Enabled() && (!HaveEL(EL3) || SCR_EL3.VTLBIDEn == '1')) && HCRX_EL2.FNB ==
        '1') then
        if IsFeatureImplemented(FEAT_XS) && IsFeatureImplemented(FEAT_HCX) &&
        IsHCRXEL2Enabled() && HCRX_EL2.FnXS == '1' then
            AArch64.TLBI_RVAA(SecurityStateAtEL(EL1), Regime_EL10, VMID[],
            Broadcast_ForcedISH, TLBIlevel_Any, TLBI_ExcludeXS, X[t, 64]);
        else
            AArch64.TLBI_RVAA(SecurityStateAtEL(EL1), Regime_EL10, VMID[],
            Broadcast_ForcedISH, TLBIlevel_Any, TLBI_AllAttr, X[t, 64]);
        else
            if IsFeatureImplemented(FEAT_XS) && IsFeatureImplemented(FEAT_HCX) &&
            IsHCRXEL2Enabled() && HCRX_EL2.FnXS == '1' then
                AArch64.TLBI_RVAA(SecurityStateAtEL(EL1), Regime_EL10, VMID[],
                Broadcast_NSH, TLBIlevel_Any, TLBI_ExcludeXS, X[t, 64]);
            else
                AArch64.TLBI_RVAA(SecurityStateAtEL(EL1), Regime_EL10, VMID[],
                Broadcast_NSH, TLBIlevel_Any, TLBI_AllAttr, X[t, 64]);
```

is changed to read:

```
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TTLB == '1' && ... then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && ... && HFGITR_EL2.TLBIRVAAE1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.TLBI_RVAA(SecurityStateAtEL(EL1), Regime_EL10, VMID[],
        Broadcast_NSH, TLBIlevel_Any, TLBI_AllAttr, X[t, 64]);
```

This change is applied to the accessibility pseudocode for the AT, DC, IC, PLBI, and TLBI system instructions.

The section **J1.1.3 aarch64/functions**, function AArch64.TLBI_RVAA(), the code that reads:

```
AArch64.TLBI_RVAA(SecurityState security, Regime regime, bits(16) vmid,
    Broadcast broadcast, TLBIlevel level, TLBIMemAttr attr, bits(64)
    Xt)
    assert PSTATE.EL IN {EL3, EL2, EL1};

    TLBIRecord r;
    ...
```

is changed to read:

```
AArch64.TLBI_RVAA(SecurityState security, Regime regime_in, bits(16) vmid_in,
    Broadcast broadcast_in, TLBIlevel level_in, TLBIMemAttr attr_in,
```

```

        bits(64) Xt)
assert PSTATE.EL IN {EL3, EL2, EL1};
Broadcast broadcast = AArch64.EffectiveBroadcast(broadcast_in);
TLBIMemAttr attr = attr_in;

if attr == TLBI_AllAttr && ExcludeXS() then
    attr = TLBI_ExcludeXS;

Regime regime = regime_in;
bits(16) vmid = vmid_in;

if ELIsInHost(EL0) then
    regime = Regime_EL20;
    vmid   = VMID_NONE;

TLBIRecord r;
...

```

In the same section, AArch64.EffectiveBroadcast() is changed to read:

```

Broadcast AArch64.EffectiveBroadcast(Broadcast broadcast_in)
    Broadcast broadcast = broadcast_in;

    if PSTATE.EL != EL1 then return broadcast;

    if IsFeatureImplemented(FEAT_TLBIID) then
        bit fnb;
        if IsHCRXEL2Enabled() && (!HaveEL(EL3) || SCR_EL3.VTLBIDEn == '1') then
            fnb = HCRX_EL2.FNB;
        else
            fnb = '0';

        case fnb:HCR_EL2.FB of
            when '00'
            when '01'
                if broadcast == Broadcast_NSH then
                    // The operation is broadcast within the Inner Shareable
                    shareability domain.
                    broadcast = Broadcast_ForcedISH;
            when '10'
                if broadcast == Broadcast_ISH then
                    // The operation only affects the PE on which the instruction is
                    executed.
                    broadcast = Broadcast_NSH;
            when '11'
                if broadcast == Broadcast_ISH then
                    // The operation is broadcast to PEs which would share TLB
                    entries with the
                    // current PE if CnP were 1.
                    broadcast = Broadcast_ISH_CnP;
        else
            if broadcast == Broadcast_NSH && EL2Enabled() && HCR_EL2.FB == '1' then
                broadcast = Broadcast_ForcedISH;

    return broadcast;

```

Additionally, ExcludeXS() is changed to read:

```

boolean ExcludeXS()
    return (IsFeatureImplemented(FEAT_XS) && PSTATE.EL == EL1 &&
            IsHCRXEL2Enabled() && HCRX_EL2.FnXS == '1');

```

2.72 D24952

In section **A2.3.7 “The Armv9.6 architecture extension”**, the feature FEAT_AMU_EXTACR is moved to **A2.3.3 “The Armv9.2 architecture extension”**

2.73 D24957

In section **J1.1.3 “aarch64/functions”**, in the pseudocode function BTypeCompatible_PAC(), the code that reads:

```
boolean BTypeCompatible_PAC(PACInstType pacinst)
...
if IsFeatureImplemented(FEAT_PAuth_EnhCtl) && pacinst == PACIxSP then
    bit btd;
    case PSTATE.EL of
        when EL0 btd = if ELIsInHost(EL0) then SCTLR2_EL2.BTD0 else
SCTLR2_EL1.BTD0;
        when EL1 btd = SCTLR2_EL1.BTD;
        when EL2 btd = SCTLR2_EL2.BTD;
        when EL3 btd = SCTLR2_EL3.BTD;
    ...
```

is changed to read:

```
boolean BTypeCompatible_PAC(PACInstType pacinst)
...
if IsFeatureImplemented(FEAT_PAuth_EnhCtl) && pacinst == PACIxSP then
    bit btd;
    case PSTATE.EL of
        when EL0
            if ELIsInHost(EL0) then
                btd = if IsSCTLR2EL2Enabled() then SCTLR2_EL2.BTD0 else '0';
            else
                btd = if IsSCTLR2EL1Enabled() then SCTLR2_EL1.BTD0 else '0';
        when EL1 btd = if IsSCTLR2EL1Enabled() then SCTLR2_EL1.BTD else '0';
        when EL2 btd = if IsSCTLR2EL2Enabled() then SCTLR2_EL2.BTD else '0';
        when EL3 btd = SCTLR2_EL3.BTD;
    ...
```

2.74 D24960

In section **D24.12.14 “MPAMSM_EL1, MPAM Streaming Mode Register”**, the text that reads:

The MPAM labels in this register are only used if MPAM1_EL1.MPAMEN (MPAM 1.0) or MPAMCTL_EL1.MPAMEN (MPAM 2.0) is 1.

is changed to read:

When FEAT_MPAMv1p0 or FEAT_MPAMvOp1 is implemented, the MPAM labels in this register are used if MPAM1_EL1.MPAMEN is 1.

In the same section, the text that reads:

This register is present only when FEAT_MPAM is implemented or FEAT_MPAMv2 is implemented. Otherwise, direct accesses to MPAM0_EL1 are **UNDEFINED**.

is changed to read:

This register is present only when FEAT_MPAM is implemented. Otherwise, direct accesses to MPAM0_EL1 are **UNDEFINED**.

Similar changes are made to the following sections:

- **D24.12.1, “MPAM0_EL1, MPAM0 Register (EL1)”**
- **D24.12.13, “MPAMIDR_EL1, MPAM ID Register (EL1)”**

In section **D24.4.2, “TRBIDR_EL1, Trace Buffer ID Register”**, the following text is removed:

MPAM2, bits [19:16]

When FEAT_TRBE_EXT is implemented:

FEAT_MPAMv2 extensions. Indicates Memory Partitioning and Monitoring (MPAM) support using FEAT_MPAMv2 in the Trace Buffer Unit when using External mode.

The value of this field is an **IMPLEMENTATION DEFINED** choice of:

MPAM2	Meaning
0b0000	Trace Buffer External Mode is not implemented or the version of MPAM supported by this field is not implemented by the PE.
0b0001	FEAT_MPAMv2 is implemented by the Trace Buffer Unit, using default PARTID and PMG values in External mode.
0b0010	Trace Buffer MPAM extensions implemented, using FEAT_MPAMv2.

When FEAT_MPAMv2 is not implemented by the PE, the only permitted value is 0b0000.

When FEAT_MPAMv2 is implemented by the PE, the value 0b0000 is not permitted.

FEAT_TRBE_MPAM implements the functionality identified by the value 0b0010.

Access to this field is RO.

Otherwise:

Reserved, **RES0**.

2.75 C25027

In section **C5.1.4.8 “Reserved encoding space for IMPLEMENTATION DEFINED instructions”**, the following text is added:

Accesses to any of the **IMPLEMENTATION DEFINED** system instruction encodings that are not trapped are permitted to be **UNDEFINED**.

Additionally, in section **D23.3.2 “Reserved encodings for IMPLEMENTATION DEFINED registers”**, the following text is added:

Accesses to these registers that are not trapped are permitted to be **UNDEFINED**.

Arm recommends that **IMPLEMENTATION DEFINED** registers that are not **UNDEFINED** follow the principles in C5.1.1 Principles of the System instruction class encoding.

2.76 C25036

In section **H7.1.1.1 “Suspending and activating PC Sample-based Profiling”**, the text that reads:

When all of the following apply, reading PMPCSR[31:0] or PMPCSR[63:0] makes PC Sample-based Profiling active:

- PC Sample-based Profiling is suspended.
- PC sampling is allowed.
- Sampling on reads is enabled.

A subsequent external read of PMPCSR will observe the updated state.

This means that if FEAT_PCSRv8p9 is implemented and PMPCSCTL.IMP reads as 1, then PMPCSCTL.EN is set to 1 as a sideeffect of the read of PMPCSR.

is changed to read:

When all of the following apply, reading PMPCSR[31:0] or PMPCSR[63:0] makes PC Sample-based Profiling active:

- PC Sample-based Profiling is suspended.
- PC sampling is allowed.
- Sampling on reads is enabled.

A subsequent external read of PMPCSR will observe the updated state.

This means that if FEAT_PCSRv8p9 is implemented and PMPCSCTL.IMP reads as 1, then PMPCSCTL.EN is set to 1 as a side-effect of the read of PMPCSR[31:0] or PMPCSR[63:0].

Note:

A read of PMPCSR[63:32] in isolation does not make PC Sample-based Profiling active, and so does not have the side-effect of setting PMPCSCTL.EN to 1.

2.77 R25037

In section **D17.8 “Profiling Buffer management”**, rule R_{HJXHT} , the text that reads:

R_{HJXHT}

If FEAT_SPE_EXC is implemented, then when a Profiling Buffer management event is generated:

- If EL3 is implemented and any of the following are true, then the management event is recorded in PMBSR_EL3:
 - MDCR_EL3.PMSEE is 0b11.
 - MDCR_EL3.PMSEE is 0b10 and the management event is due to a fault on a write to the Profiling Buffer that would generate a Data Abort exception taken to EL3 if generated by a store instruction executed at the owning Exception level. That is, any of the following faults on a write to the Profiling Buffer:
 - Granule Protection Check (GPC) faults, other than Granule Protection Faults (GPFs). For more information, see Granule Protection Check faults.
 - If SCR_EL3.GPF is 0b1, GPFs.
 - If SCR_EL3.EA is 0b1, External aborts. See also I_{FDVRZ} .
- Otherwise, if all of the following are true, then the management event is recorded in PMBSR_EL2:
 - EL3 is not implemented or MDCR_EL3.PMSEE is not 0b00.
 - EL2 is implemented and any of the following are true:
 - The Effective value of SCR_EL3.NS is 0b1.
 - The Effective value of SCR_EL3.{NS, EEL2} is {0b0, 0b1}. This applies even when the current Exception level is EL3.
 - One of the following is true:
 - PMSCR_EL2.EE is 0b11.
 - PMSCR_EL2.EE is 0b10 and the management event is due to any fault on a write to the Profiling Buffer that would generate a Data Abort exception taken to EL2 if generated by a store instruction executed at the owning Exception level. If the owning Exception level is EL2, this means any fault on a write to the Profiling Buffer. If the owning Exception level is EL1, this means any of the following faults on a write to the Profiling Buffer:
 - Stage 2 faults.
 - If HCR_EL2.GPF is 0b1, GPFs.
 - If HCR_EL2.TEA is 0b1, synchronous External aborts. See also I_{FDVRZ} .

- PMSCR_EL2.EE is 0b10 and the management event is due to a GPC fault other than a GPF.
- Otherwise, the management event is recorded in PMBSR_EL1.

is changed to read:

R_{HJXHT}

If FEAT_SPE_EXC is implemented, then when a Profiling Buffer management event is generated:

- If EL3 is implemented and any of the following are true, then the management event is recorded in PMBSR_EL3:
 - MDCR_EL3.PMSEE is 0b11.
 - MDCR_EL3.PMSEE is 0b10 and the management event is due to a fault on a write to the Profiling Buffer that would generate a Data Abort exception taken to EL3 if generated by a store instruction executed at the owning Exception level. That is, any of the following faults on a write to the Profiling Buffer:
 - Granule Protection Check (GPC) faults, other than Granule Protection Faults (GPFs). For more information, see Granule Protection Check faults.
 - If SCR_EL3.GPF is 0b1, GPFs.
 - If SCR_EL3.EA is 0b1, External aborts. See also I_{FDVRZ}.
 - MDCR_EL3.PMSEE is 0b10 and the management event is an **IMPLEMENTATION DEFINED** buffer management event that the implementation elects to record in PMBSR_EL3.
- Otherwise, if all of the following are true, then the management event is recorded in PMBSR_EL2:
 - EL3 is not implemented or MDCR_EL3.PMSEE is not 0b00.
 - EL2 is implemented and any of the following are true:
 - The Effective value of SCR_EL3.NS is 0b1.
 - The Effective value of SCR_EL3.{NS, EEL2} is {0b0, 0b1}. This applies even when the current Exception level is EL3.
 - One of the following is true:
 - PMSCR_EL2.EE is 0b11.
 - PMSCR_EL2.EE is 0b10 and the management event is due to any fault on a write to the Profiling Buffer that would generate a Data Abort exception taken to EL2 if generated by a store instruction executed at the owning Exception level. If the owning Exception level is EL2, this means any fault on a write to the Profiling Buffer. If the owning Exception level is EL1, this means any of the following faults on a write to the Profiling Buffer:
 - Stage 2 faults.
 - If HCR_EL2.GPF is 0b1, GPFs.
 - If HCR_EL2.TEA is 0b1, synchronous External aborts. See also I_{FDVRZ}.

- PMSCR_EL2.EE is 0b10 and the management event is due to a GPC fault other than a GPF.
- PMSCR_EL2.EE is 0b10 and the management event is an **IMPLEMENTATION DEFINED** buffer management event that the implementation elects to record in PMBSR_EL2.
- Otherwise, the management event is recorded in PMBSR_EL1.

A similar relaxation is made to rule R_{BNQRP} in section **D6.5 “Trace buffer management”**.

2.78 D25054

In section **J1.1.4 “aarch64/translation”**, the code that reads:

```
boolean AArch64.SettingAccessFlagPermitted(FaultRecord fault)
{
    if fault.statuscode == Fault_None then
        return TRUE;
    elseif fault.statuscode IN {Fault_Alignment, Fault_Permission} then
        return ConstrainUnpredictableBool(Unpredictable_AFUPDATE);
    else
        return FALSE;
}
```

is changed to read:

```
boolean AArch64.SettingAccessFlagPermitted(FaultRecord fault, FaultRecord
    fault_perm)
{
    if fault.statuscode == Fault_None && fault_perm.statuscode == Fault_None then
        return TRUE;
    elseif fault.statuscode == Fault_Alignment || fault_perm.statuscode ==
        Fault_Permission then
        return ConstrainUnpredictableBool(Unpredictable_AFUPDATE);
    else
        return FALSE;
}
```

2.79 D25060

In section **D24.2.62 “HCR_EL2, Hypervisor Configuration Register”**, in the description of field ‘TIDCP, bit [20]’, the text that reads:

TIDCP, bit [20]

Trap **IMPLEMENTATION DEFINED** functionality. Traps EL1 accesses to the encodings reserved for **IMPLEMENTATION DEFINED** functionality to EL2, when EL2 is enabled in the current Security state as follows:

- In AArch64 state, ELO access to the encodings in the following reserved encoding spaces are trapped:
 - **IMPLEMENTATION DEFINED** System instructions, which are accessed using SYS and SYSL, with CRn == {11, 15}, and are reported using EC syndrome value 0x18.

- **IMPLEMENTATION DEFINED** System instructions, which are accessed using SYSP, with CRn == {11, 15}, and are reported using EC syndrome value 0x14.
- **IMPLEMENTATION DEFINED** System registers, which are accessed using MRS and MSR with the S3_<op1>_<Cn>_<Cm>_<op2> register name, and are reported using EC syndrome 0x18.
- **IMPLEMENTATION DEFINED** System registers, which are accessed using MRRS and MSRR with the S3_<op1>_<Cn>_<Cm>_<op2> register name, and are reported using EC syndrome 0x14.
- In AArch32 state, MCR and MRC access to instructions with the following encodings are trapped and reported using EC syndrome value 0x03:
 - All coproc==p15, CRn==c9, opc1 == {0-7}, CRm == {c0-c2, c5-c8}, opc2 == {0-7}.
 - All coproc==p15, CRn==c10, opc1 == {0-7}, CRm == {c0, c1, c4, c8}, opc2 == {0-7}.
 - All coproc==p15, CRn==c11, opc1 == {0-7}, CRm == {c0-c8, c15}, opc2 == {0-7}.

When this functionality is accessed from EL0:

- If FEAT_TIDCP1 is implemented and the Effective value of SCTLR_EL1.TIDCP is 1, any accesses from EL0 are trapped to EL1.
- Otherwise, if FEAT_TIDCP1 is implemented and the Effective value of SCTLR_EL2.TIDCP is 1, any accesses from EL0 are trapped to EL2.
- Otherwise:
 - If HCR_EL2.TIDCP is 1, it is **IMPLEMENTATION DEFINED** whether any accesses from EL0 are trapped to EL2.
 - If HCR_EL2.TIDCP is 0, any accesses from EL0 are **UNDEFINED** and generate an exception that is taken to EL1 or EL2.

is changed to read:

TIDCP, bit [20]

Trap **IMPLEMENTATION DEFINED** functionality. Traps EL1 accesses to the encodings reserved for **IMPLEMENTATION DEFINED** functionality to EL2, when EL2 is enabled in the current Security state as follows:

- In AArch64 state, EL1 access to the encodings in the following reserved encoding spaces are trapped:
 - **IMPLEMENTATION DEFINED** System instructions, which are accessed using SYS and SYSL, with CRn == {11, 15}, and are reported using EC syndrome value 0x18.
 - **IMPLEMENTATION DEFINED** System instructions, which are accessed using SYSP, with CRn == {11, 15}, and are reported using EC syndrome value 0x14.
 - **IMPLEMENTATION DEFINED** System registers, which are accessed using MRS and MSR with the S3_<op1>_<Cn>_<Cm>_<op2> register name, and are reported using EC syndrome 0x18.

- **IMPLEMENTATION DEFINED** System registers, which are accessed using MRRS and MSRR with the S3_<op1>_<Cn>_<Cm>_<op2> register name, and are reported using EC syndrome value 0x14.
- In AArch32 state, MCR and MRC access to instructions with the following encodings are trapped and reported using EC syndrome value 0x03:
 - All coproc==p15, CRn==c9, opc1 == {0-7}, CRm == {c0-c2, c5-c8}, opc2 == {0-7}.
 - All coproc==p15, CRn==c10, opc1 == {0-7}, CRm == {c0, c1, c4, c8}, opc2 == {0-7}.
 - All coproc==p15, CRn==c11, opc1 == {0-7}, CRm == {c0-c8, c15}, opc2 == {0-7}.

For accesses to these regions of encoding space by EL0, all of the following apply:

- If FEAT_TIDCP1 is implemented, traps caused by SCTLRL_EL1.TIDCP and SCTLRL_EL2.TIDCP are reported with higher priority than traps caused by this control.
- If FEAT_TIDCP1 is not implemented, or the access is not trapped by SCTLRL_ELx.TIDCP, one of the following applies:
 - If HCR_EL2.TIDCP is 0 then it is **IMPLEMENTATION DEFINED**, for each encoding, whether the access is performed, is **UNDEFINED**, or is trapped by an **IMPLEMENTATION DEFINED** mechanism.
 - If HCR_EL2.TIDCP is 1 then it is **IMPLEMENTATION DEFINED**, for each encoding, whether the access is trapped to EL2. If it is not trapped it is **UNDEFINED**.

3. Arm Memory System Resource Partitioning and Monitoring (MPAM) System Component Specification

This section records known issues in the Arm Memory System Resource Partitioning and Monitoring (MPAM) System Component Specification (IHI 0099), Issue B.a.

No known issues at this time.

4. Arm Reliability, Availability, and Serviceability (RAS) System Architecture for A-profile architecture

This section records known issues in the Arm Reliability, Availability, and Serviceability (RAS) System Architecture for A-profile architecture (IHI 0100), Issue A.b.

4.1 D24969

In section **3.2.1 “ERRACR, Access Configuration Register”**, in the descriptions of fields ‘RLRA’, ‘SRA’, and ‘NSRA’, the following text is removed:

Note

Realm access to error records is enabled from reset.